Project 3

2024년 2학기	
제출일	2024. 12. 15.
과목명	데이터베이스및데이터시각화
담당교수	이기훈

학과	컴퓨터정보공학부	학과	컴퓨터정보공학부
학번	2021202022	학번	2021202039
이름	한은진	이름	이소영
학과		학과	
학번		학번	
이름		이름	

KWANGWOON UNIVERSITY

1. 웹 애플리케이션에 대한 설명

인터넷 중고 장터 플랫폼인 '가지마켓'을 구현하였습니다. 이 프로젝트는 사용자 간 원활한 중고 거래를 지원하는 것을 목표로 하며, 데이터베이스에 저장된 다양한 데이터를 활용하여 기능을 제공하고 있습니다. 실제 사용자의 입장에서 효율적인 거래 경험을 제공할 수 있도록 애플리케이션 개발에 중점을 두었습니다.

주요 기능은 인터넷 중고 장터에 필요한 필수 기능과 사용자 편의성을 높이고 데이터를 다양하게 활용할 수 있는 특별 기능으로 구성됩니다.

기본 기능

No	기능 종류	
1	회원가입	
2	로그인	
3	상품 조회	
4	상품 문의	
5	상품 후기 작성	
6	게시글 작성	
7	상품 검색	
8	판매 중인 상품/판매 내역 확인	
9	데이터 시각화	관리자용 카테고리별 상품 수 데이터
		관리자용 최신 일주일 일별 포인트 충전 추이

특별 기능

No	기능 종류
1	매너 지수 시스템
2	프로필 사진 및 소개글 설정
3	인기순/최신순 상품 조회
4	상품별 키워드 시스템
5	키워드를 이용한 관련 상품 추천
6	상품 찜 기능
7	채팅 기능
8	포인트 충전 및 인출 기능
9	채팅 중 포인트 송금 기능
10	관리자 계정 관리
11	중요 공지 설정
12	상품 게시글 및 판매자 신고 기능
13	사용자 회원 영구 정지
14	익명 문의 기능

프로젝트 구현 환경은 다음과 같습니다.

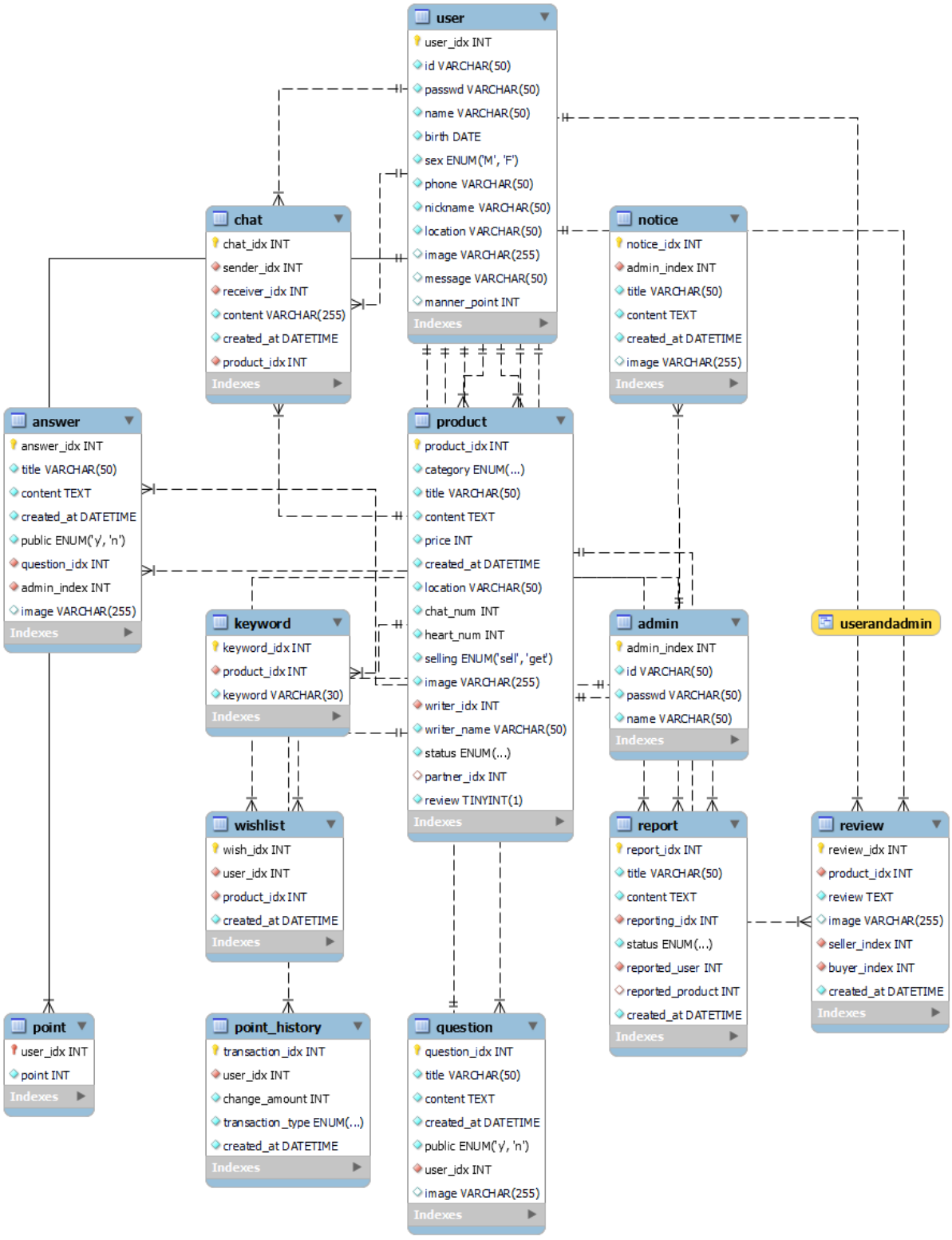
본 프로젝트는 3-Tier Architecture 기반으로 설계되었으며, 프레젠테이션 레이어(이하 프론트엔드), 비즈니스 로직 레이어(이하 백엔드), 데이터 레이어(이하 데이터베이스)의 세 가지 계층으로 구성되어 있습니다.

- 프론트엔드: 사용자 인터페이스는 React와 JavaScript를 사용하여 구현합니다.
- 백엔드: Spring Boot와 Java를 사용하여 비즈니스 로직을 구현합니다.
- 데이터베이스: 모든 사용자, 상품, 거래 관련 데이터를 MySQL 데이터베이스에 저장하고 관리합니다.

이 구조를 통해 각 계층이 독립적으로 유지되며, 효율적이고 확장 가능한 애플리케이션 개발을 목표로 하고 있습니다.

Database	Server	Client Interface	Language
			 

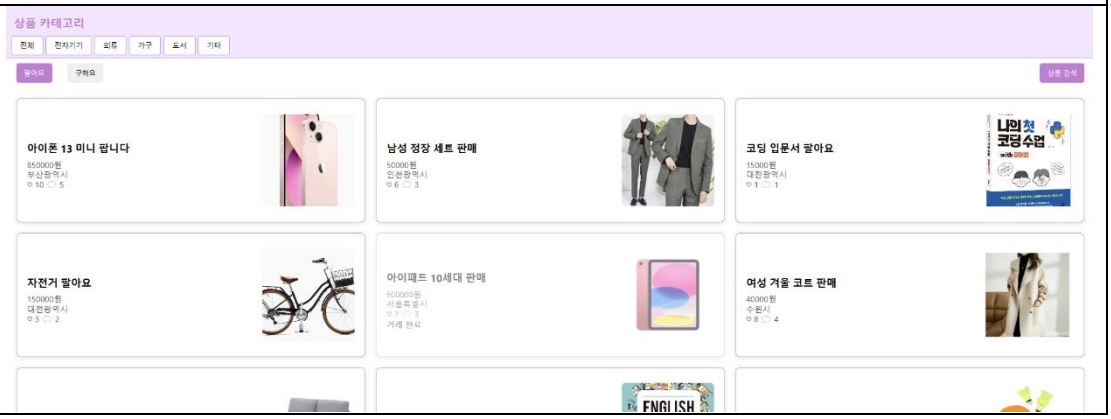
다음은 ER diagram입니다.



2. 주요 기능 설명

주요 기능은 각 기능 별로 표를 생성하여 설명해 두었습니다.

기능	**상품 목록 조회**
기능	전체 상품 목록을 조회할 수 있습니다. 상품들은 간단한 정보만 담은 카드 형태로 표시

설명	되는데, 각 상품 카드를 클릭하면 상품 게시글로 이동할 수 있습니다. 상품 카드에는 제목, 가격, 거래 지역, 찜 수, 채팅 수, 이미지가 표시됩니다. 카테고리별 페이지에서는 최신순으로 상품을 정렬합니다.
기능 구현 방식 (API, SQL code 포함)	<API> /product <SQL code> SELECT * FROM product WHERE selling = ? AND status != 'removed' ORDER BY created_at DESC
결과 화면	

기능	**상품 게시글 조회**
기능 설명	상품 게시글에는 카테고리, 제목, 가격, 거래 지역, 게시 날짜, 거래 상태(거래 중, 삭제, 거래 완료), 거래 종료(팔아요, 구해요), 관심 수, 채팅 수, 작성자 닉네임 정보가 표시됩니다. 삭제된 글은 관리자만 확인할 수 있습니다. 상품 정보 아래에는 찜, 채팅, 신고, 삭제, 리뷰 작성 버튼이 있습니다. 게시글 작성자는 찜, 삭제 버튼을 볼 수 있고, 관리자는 삭제 버튼을 볼 수 있으며, 기타 사용자는 찜, 채팅, 신고 버튼을 볼 수 있습니다. 버튼 하단에는 키워드 기반 추천 상품 섹션이 있으며, 같은 키워드를 가진 다른 상품들이 카드 형태로 표시됩니다.
기능 구현 방식 (API, SQL code 포함)	<API> /product/{product_idx} <SQL code> 1. 상품 정보 조회 SELECT p.*, CASE WHEN ? IS NOT NULL AND w.wish_idx IS NOT NULL THEN TRUE ELSE FALSE END AS is_hearted FROM product p LEFT JOIN wishlist w ON p.product_idx = w.product_idx AND w.user_idx = ? WHERE p.product_idx = ?

2. 키워드 조회
```
SELECT keyword FROM keyword WHERE product_idx = ?
```

3. 추천 상품 조회
```
SELECT DISTINCT
    sub.*,
    CASE
        WHEN ? IS NOT NULL AND EXISTS (
            SELECT 1
            FROM wishlist w
            WHERE w.product_idx = sub.product_idx AND w.user_idx = ?
        ) THEN TRUE
        ELSE FALSE
    END AS is_hearted
FROM (
    SELECT
        p.*,
        k.keyword
    FROM
        product p
    JOIN
        keyword k ON p.product_idx = k.product_idx
    WHERE
        k.keyword IN (%s)
        AND p.product_idx != ?
        AND p.status = 'active'
) AS sub;
```

결과 화면



카테고리 > 의류

여성 겨울 코트 판매

40,000원
수원시 / 2024. 12. 10.
거래 중 / 구해요
♡ 관심 8 · 💬 채팅 4

작성자 길동이

사이즈 M, 따뜻하고 가벼워요.

찜♥ 삭제 수정

키워드 기반 추천 상품

남성 정장 세트 판매

50000원
인천광역시
♡ 6 💬 3



여성 자켓 팝니다

25000원
대전광역시
♡ 4 💬 2

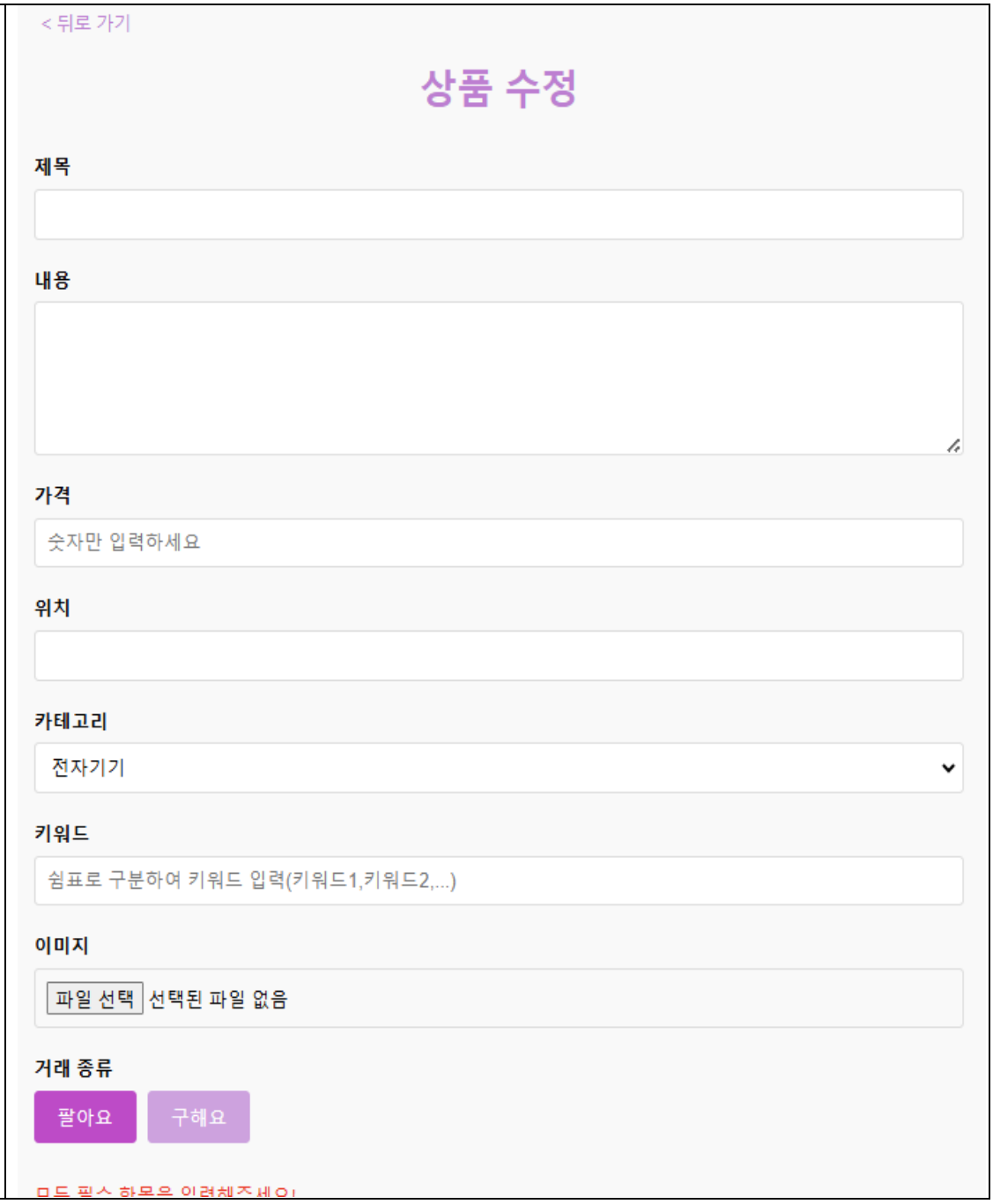


기능	상품 게시글 작성(상품 등록)
기능 설명	로그인한 회원은 상품을 게시할 수 있습니다. 제목, 내용, 가격, 위치, 카테고리, 키워드, 이미지를 입력하고 '팔아요, 구해요' 중에 거래 종류를 선택합니다. 입력된 정보는 서버로 보내져 데이터베이스 Product 테이블에 관련 정보가 저장됩니다. 이렇게 게시글이 등록되고, 상품 목록에 추가됩니다.
기능 구현 방식 (API, SQL code 포함)	<API> /product/upload <SQL code> 1. 상품 삽입 쿼리 INSERT INTO product (title, content, image, price, category, location, created_at, writer_idx, writer_name, selling, status) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?) 2.마지막 삽입된 product_idx 조회 SELECT LAST_INSERT_ID() 3. 키워드 삽입 쿼리 INSERT INTO keyword (product_idx, keyword) VALUES (?, ?)

결과 화면	

기능	상품 게시글 수정
기능 설명	상품 게시글의 작성자는 상품을 수정할 수 있습니다. 제목, 내용, 가격, 위치, 카테고리, 키워드, 이미지를 다시 입력하고 '팔아요, 구해요' 중에 거래 종류를 다시 선택할 수 있습니다. Product 테이블에서 해당 상품 정보를 찾아 업데이트합니다.
기능 구현 방식 (API, SQL code 포	<API> /product/{product_idx}/edit <SQL code> UPDATE product SET title = ?, content = ?,

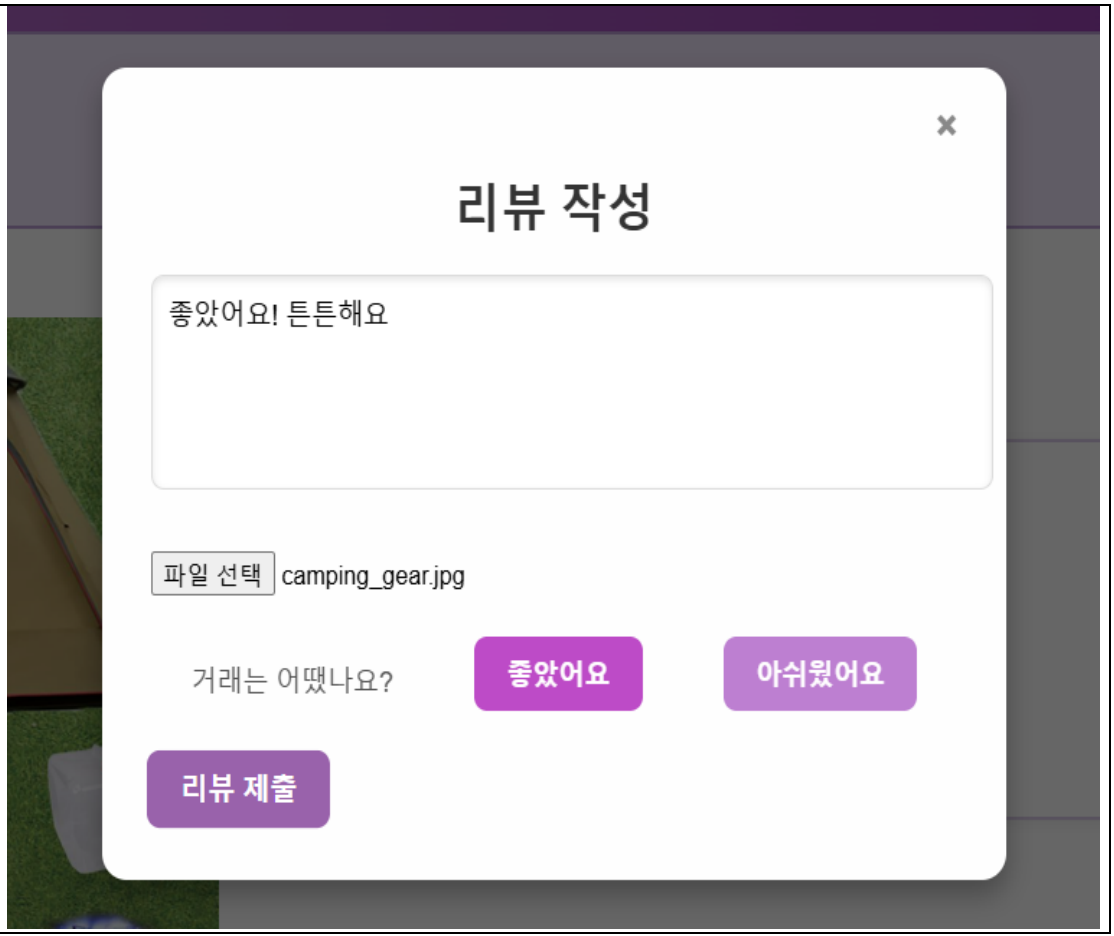
함)	image = ?, price = ?, category = ?, location = ?, created_at = ?, writer_idx = ?, writer_name = ?, selling = ?, status = ? WHERE product_idx = ? DELETE FROM keyword WHERE product_idx = ? INSERT INTO keyword (product_idx, keyword) VALUES (?, ?)

결과 화면	

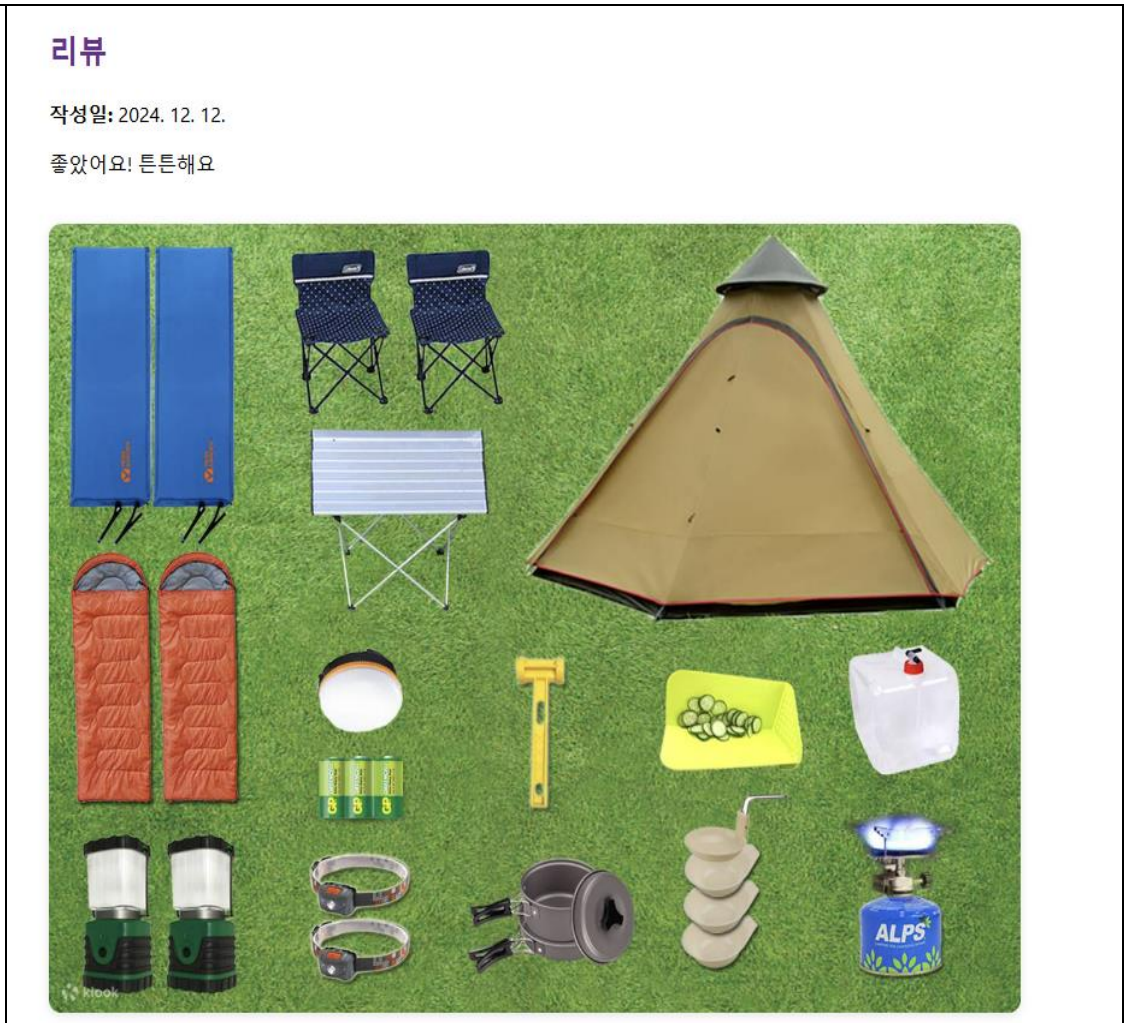
기능	상품 게시글 삭제
기능 설명	회원은 자신이 작성한 상품 게시글을 삭제할 수 있습니다. 실제로 데이터베이스에서 삭제되는 게 아니라 Product 테이블에서 상태 정보 status가 삭제 상태로 업데이트됩니다.
기능 구현 방식 (API, SQL code 포함)	<API> /product/{product_idx}/delete <SQL code> UPDATE product SET status = 'removed' WHERE product_idx = ?

결과 화면	삭제 결과 화면은 따로 없습니다.

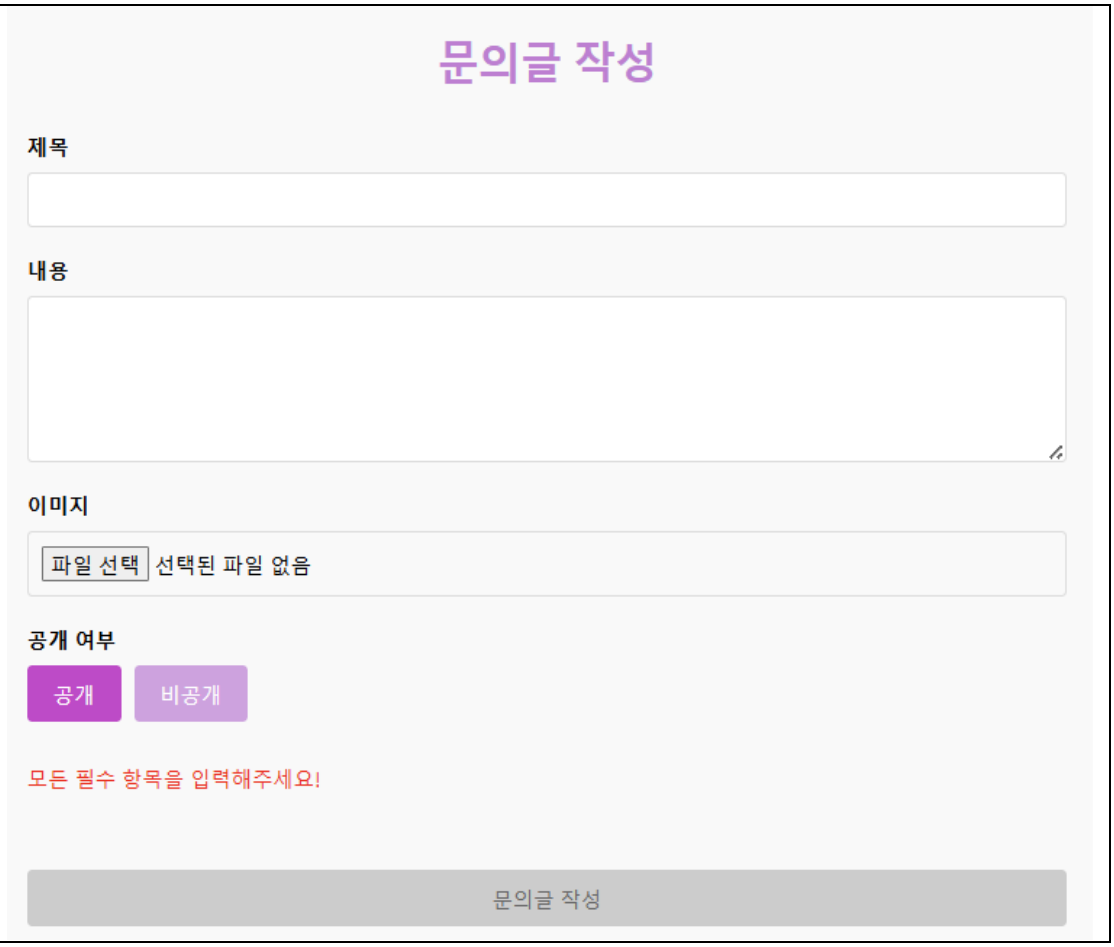
기능	상품 후기 작성
기능 설명	해당 상품을 거래한 상대방은 상품 게시글에서 리뷰 작성 버튼을 통해 후기를 작성할 수 있습니다. 리뷰 내용과 이미지, 거래 평가(좋았어요, 아쉬웠어요)를 입력합니다. 회원은 거래한 상품에 대해 리뷰를 작성할 수 있습니다. 리뷰 내용을 작성하고, 사진을 첨부할 수 있으며 상품 게시자에 대해 평가합니다. “좋았어요”를 선택하면 게시자의 매너 지수가 5 상승하고, “아쉬웠어요”를 선택하면 5 하강합니다.
기능 구현 방식 (API, SQL code 포함)	<API> /product/{product_idx}/review/write <SQL code> 1. 리뷰 삽입 INSERT INTO Review (product_idx, review, image, seller_index, buyer_index, created_at) VALUES (?, ?, ?, ?, ?, ?) 2. 상품 테이블에서 리뷰 상태 업데이트 UPDATE Product SET review = true WHERE product_idx = ? 3. 판매자의 매너 포인트 업데이트 (User 테이블) UPDATE user SET manner_point = manner_point + CASE WHEN ? = 'good' THEN 5 WHEN ? = 'bad' THEN -5 ELSE 0 END WHERE user_idx = ?

결과 화면	

기능	상품 후기 조회
기능 설명	등록된 리뷰는 상품 게시글 하단에 추가되어 표시됩니다. 리뷰는 Review 테이블에서 조회 가능합니다.
기능 구현 방식 (API, SQL code 포함)	<API> /product/{id}/review/read <SQL code> SELECT r.review_idx, r.product_idx, r.review, r.image, r.seller_index, r.buyer_index, r.created_at FROM Review r WHERE r.product_idx = ?

결과 화면	

기능	문의 게시글 작성
기능 설명	로그인한 회원은 문의글을 작성하고, 관리자는 문의글에 답글을 작성할 수 있습니다. 회원의 문의글 작성 시에 제목, 내용, 이미지, 공개(공개, 비공개)를 입력하여 제출하면 Question 테이블에 문의사항으로 등록됩니다.
기능 구현 방식 (API, SQL code 포함)	<API> /ask/write <SQL code> INSERT INTO Question (title, content, created_at, public, user_idx, image) VALUES (?, ?, NOW(), ?, ?, ?)

결과 화면	

기능	문의 목록 조회
기능 설명	문의사항 게시판에서 문의글을 확인할 수 있습니다. 순번, 제목, 날짜, 공개여부, 문의글/답글 여부가 표시되며 각 글의 제목을 클릭하면 상세 내용을 확인할 수 있습니다.
기능 구현 방식 (API, SQL code 포함)	<API> /ask <SQL code> SELECT q.question_idx, q.title, q.content, q.created_at, q.public, q.user_idx, q.image, u.nickname FROM Question q JOIN User u ON q.user_idx = u.user_idx WHERE q.question_idx = ?

결과 화면	

기능	문의 게시글 조회
기능 설명	해당 글의 제목, 작성자, 작성일, 공개여부, 내용, 첨부된 사진을 확인할 수 있습니다. 관리자는 답글 작성 버튼을 추가로 볼 수 있습니다. 문의글을 조회할 때 답글이 존재하면 답글 제목과 링크가 표시되며, 문의 답글을 조회할 때는 문의글 제목과 링크가 표시됩니다.
기능 구현 방식 (API, SQL code 포함)	<API> /ask/question/{question_idx}, /ask/answer/{answer_idx} <SQL code> SELECT q.question_idx, q.title, q.content, q.created_at, q.public, q.user_idx, q.image, u.nickname FROM Question q JOIN User u ON q.user_idx = u.user_idx WHERE q.question_idx = ? SELECT a.answer_idx, a.title, a.content, a.created_at, a.admin_index, a.image, a.public FROM Answer a WHERE a.question_idx = ?

결과 화면	

기능	문의 답글 작성
기능 설명	로그인한 회원은 문의글을 작성하고, 관리자는 문의글에 답글을 작성할 수 있습니다.관리자의 답글 작성 시에는 제목, 내용, 이미지, 공개(공개, 비공개)를 설정하면 Answer 테이블에 답글로 등록됩니다.
기능 구현 방식 (API, SQL code 포함)	<API> /ask/{question_idx}/answer <SQL code> INSERT INTO Answer (title, content, created_at, public, question_idx, admin_index, image) VALUES (?, ?, NOW(), ?, ?, ?, ?)

결과 화면	

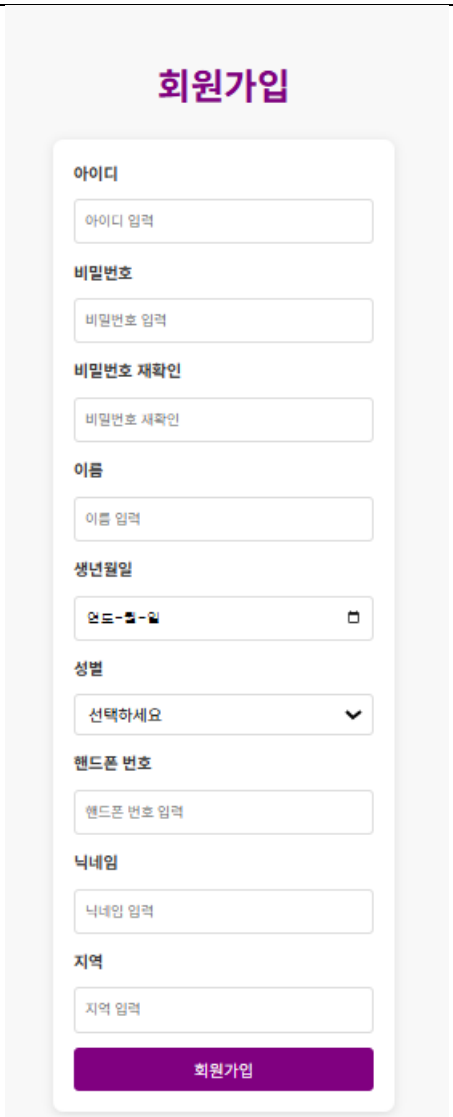
기능	**회원 목록 조회**
기능 설명	관리자는 회원 관리 페이지에서 전체 회원 목록을 조회할 수 있습니다. User 테이블에서 모든 회원에 대해 필요한 정보, ID, 닉네임, 매너지수를 가져옵니다. 각 닉네임을 클릭하면 회원 프로필로 이동할 수 있으며 회원 관리 페이지에서 매너 지수 수정과 영구 정지가 가능합니다. 회원의 매너 지수에 따라 하트의 색이 변하는 효과를 넣어 정보 전달성을 높였습니다.
기능 구현 방식 (API, SQL code 포함)	<API> /admin/get/userList <SQL code> SELECT user_idx, id, nickname, manner_point FROM user;

결과 화면	

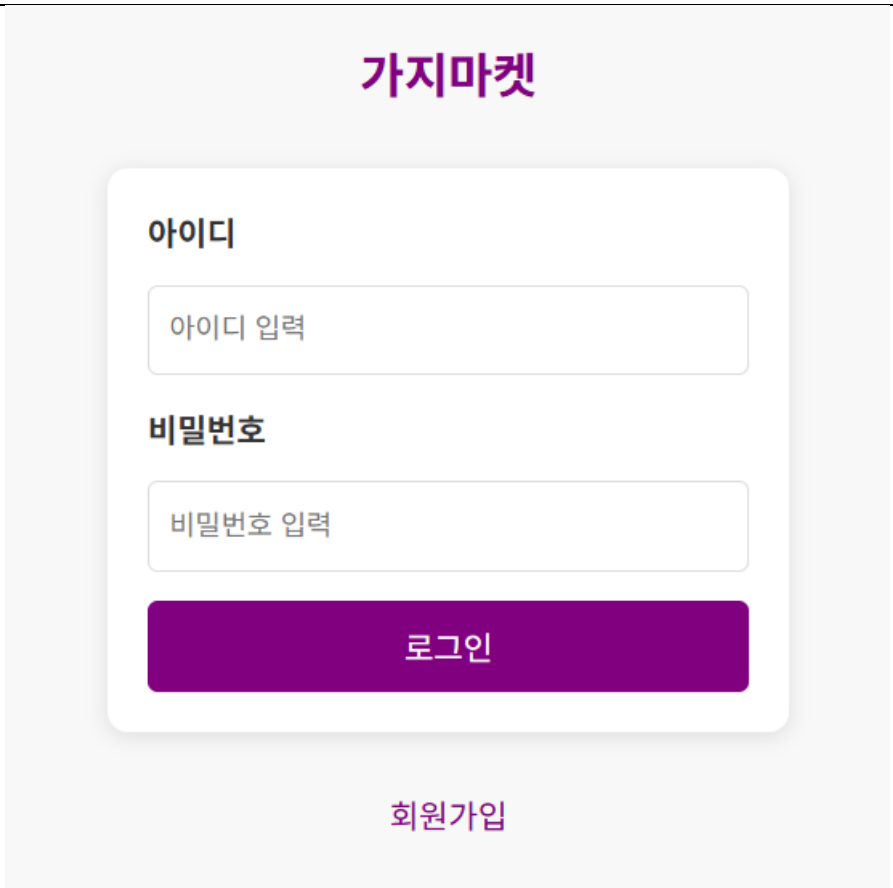
기능	**회원 영구 정지**
기능 설명	관리자는 회원 관리 페이지에서 특정 회원을 영구 정지 처리할 수 있습니다. 해당 회원은 Product 테이블에서 매너 지수가 -1로 업데이트되며 활동이 제한됩니다.
기능 구현 방식 (API, SQL code 포함)	<API> /admin/ban/{user_idx} <SQL code> UPDATE user SET manner_point = -1 WHERE user_idx = ?;
결과 화면	

기능	**상품 삭제**
기능 설명	관리자는 모든 상품 게시글을 삭제할 수 있습니니다. 실제로 데이터베이스에서 삭제되는 게 아니라 Product 테이블에서 상태 정보 status가 삭제 상태로 업데이트됩니다. 삭제된 상품 게시글은 관리자만 조회할 수 있습니다.
기능 구현 방식 (API, SQL code 포함)	<API> /product/{product_idx}/delete <SQL code> UPDATE product SET status = 'removed'

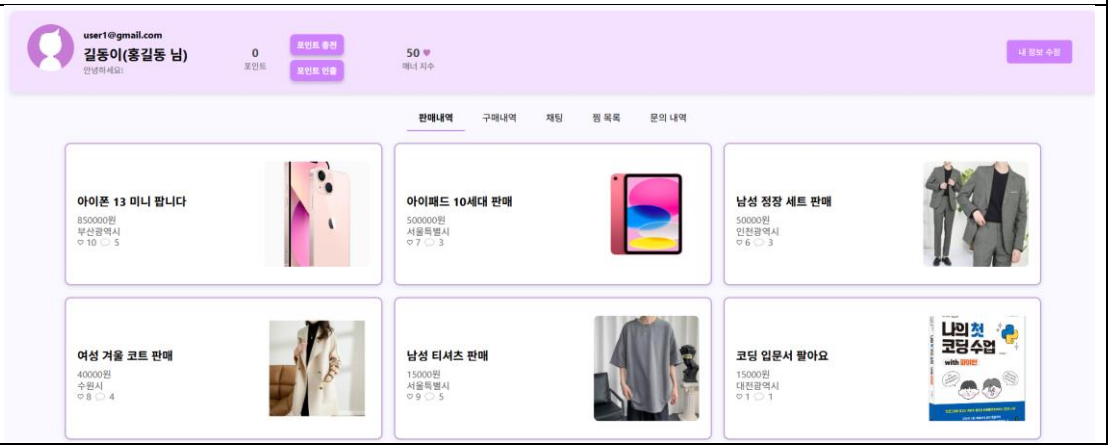
	WHERE product_idx = ?
결과 화면	상품 삭제 결과 화면은 따로 없습니다.

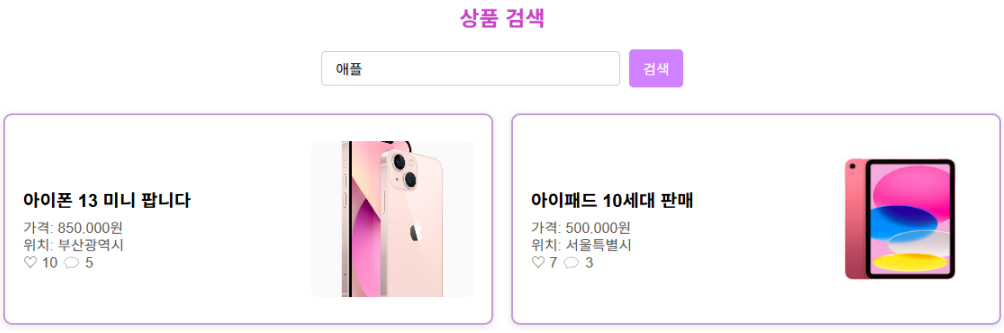
기능	**회원 가입**
기능 설명	회원 가입 기능은 사용자가 가지마켓 플랫폼의 계정을 생성할 수 있도록 합니다. 사용자는 아이디, 비밀번호, 이름, 생년월일, 성별, 전화번호, 닉네임, 지역, 프로필 이미지, 자기소개 메시지 등을 입력해야 하며, 초기 매너 점수는 기본값으로 설정됩니다. 이 정보들은 데이터베이스에 저장되며, 가입이 완료되면 사용자는 플랫폼의 다양한 기능을 사용할 수 있습니다. 데이터의 유효성 검사는 서버 측에서 처리됩니다.
기능 구현 방식 (API, SQL code 포함)	<API> /user/signup <SQL code> INSERT INTO user (id, passwd, name, birth, sex, phone, nickname, location, image, message, manner_point) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
결과 화면	

기능	**로그인**
기능 설명	로그인 기능은 사용자가 아이디와 비밀번호를 입력하여 가지마켓 플랫폼에 접

	근할 수 있도록 합니다. 입력된 아이디와 비밀번호를 데이터베이스에서 확인하고, 일치하는 사용자가 있으면 해당 사용자의 역할(role)과 매너 점수(manner_point)를 가져옵니다. 역할 정보는 일반 사용자와 관리자를 구분하는 데 사용되며, 매너 점수는 사용자 경험 향상을 위해 추가적으로 제공될 수 있습니다. 로그인 성공 시, 사용자는 자신의 계정과 관련된 기능에 접근할 수 있으며, 실패 시 오류 메시지가 표시됩니다.
기능 구현 방식 (API, SQL code 포함)	<API> /user/signin <SQL code> SELECT role, manner_point FROM UserAndAdmin WHERE id = ? AND passwd = ?
결과 화면	

기능	**(프로필에서) 판매 중인 상품 및 판매 내역 확인**
기능 설명	이 기능은 사용자가 자신의 프로필에서 현재 판매 중인 상품과 과거 판매 완료된 상품 내역을 확인할 수 있도록 합니다. 사용자는 자신의 ID를 기반으로 데이터베이스에서 작성자(writer_idx)와 연관된 상품 정보를 조회할 수 있습니다. 조회된 상품 정보는 상품 제목, 가격, 지역, 좋아요 수(heart_num), 채팅 수(chat_num), 대표 이미지 등으로 구성됩니다. 이를 통해 사용자는 자신의 판매 현황을 한눈에 확인할 수 있습니다.
기능 구현 방식 (API,	<API> /user/{email}/selling <SQL code> SELECT product_idx, title, price, location, heart_num, chat_num, image

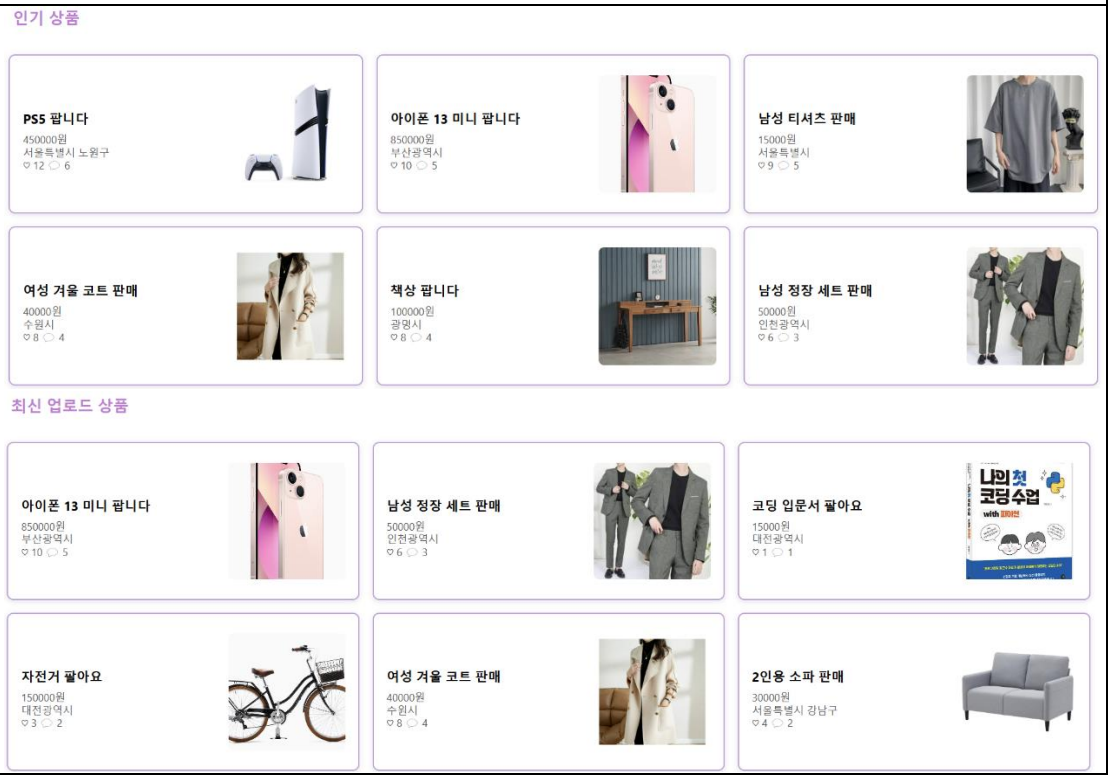
SQL code 포함)	FROM product WHERE writer_idx = (SELECT user_idx FROM user WHERE id = ?)
결과 화면	

기능	상품 검색
기능 설명	상품 검색 기능은 사용자가 입력한 검색어를 기반으로 데이터베이스에서 관련 상품을 조회할 수 있도록 합니다. 검색 대상은 상품 제목과 키워드입니다. 사용자가 검색어를 입력하면, 상품 제목(title)에 해당 검색어가 포함되거나 키워드(keyword)가 일치하는 상품이 조회됩니다. 결과는 중복 없이 반환되며, 검색된 상품 목록은 최신순으로 정렬되어 사용자에게 표시됩니다.
기능 구현 방식 (API, SQL code 포함)	<API> /product/search <SQL code> SELECT DISTINCT p.* FROM Product p LEFT JOIN Keyword k ON p.product_idx = k.product_idx WHERE p.title LIKE ? OR k.keyword LIKE ?
결과 화면	

3. 특별 기능 설명

특별 기능도 기본 기능과 마찬가지로 표 형식으로 설명해 두었습니다.

기능	인기순/최신순/카테고리/거래 타입 조건별 상품 조회
기능 설명	메인 페이지에서 상품 목록 조회 시 사용자가 현재 인기 있는 상품과 최신에 등록된 상품을 쉽게 찾을 수 있도록 정렬하는 기능입니다. Product 테이블에 있는 모든 상품 정보를 인기순은 관심(찜) 수에 따라 결정되고, 최신순은 등록 시간을 기준으로 정렬됩니다. 사용자가 다양한 상품을 탐색할 때 편의성을 높여주고, 인기 상품을 통해 소비 경향을 파악할 수 있도록 돕습니다. 또한 새로운 상품을 빠르게 보여주어 거래가 활발히 이루

	어지도록 유도합니다.
기능 구현 방식 (API, SQL code 포함)	<API> /product <SQL code> SELECT * FROM product WHERE selling = ? AND status != 'removed' ORDER BY heart_num DESC SELECT * FROM product WHERE selling = ? AND status != 'removed' ORDER BY created_at DESC
결과 화면	

기능	**상품별 키워드 시스템**
기능 설명	각 상품에는 관련된 키워드를 할당하여 검색 기능을 향상시킬 수 있으며, 이와 연계된 관련 상품 추천 기능을 통해 거래를 적극적으로 촉진할 수 있습니다. 키워드는 사용자가 상품을 등록할 때 함께 입력하게 됩니다.
기능 구현 방식 (API, SQL code 포함)	<API> /product/upload <SQL code> INSERT INTO keyword (product_idx, keyword) VALUES (?, ?)
결과 화면	해당 기능의 결과 화면은 따로 없습니다.

기능	**키워드를 이용한 관련 상품 추천**
기능 설명	상품의 키워드를 분석하여 관련 상품을 추천하는 기능입니다. 사용자가 특정 상품을 조회할 때, 해당 상품과 유사하거나 관련 있는 다른 상품들을 함께 제안하여 사용자가 만족스러운 제품을 찾을 수 있도록 돕습니다.
기능	<API>

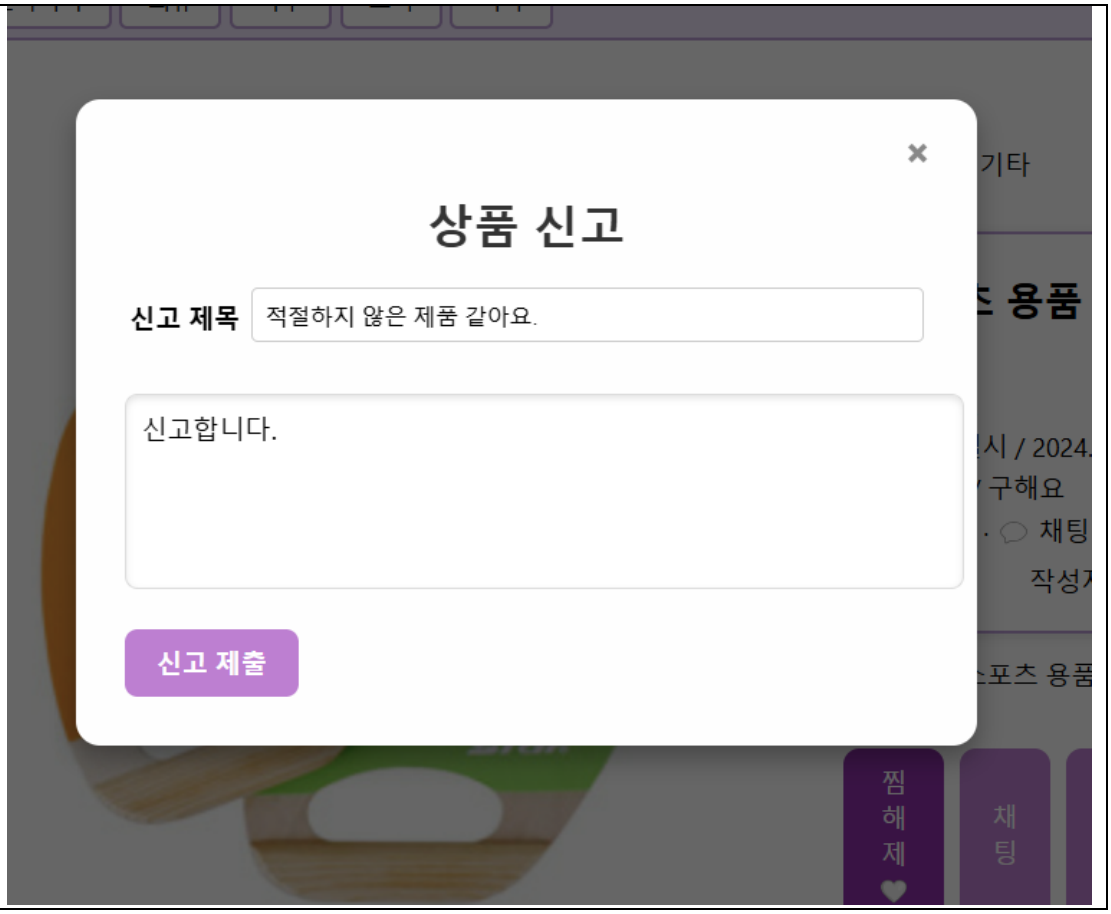
구현 방식 (API, SQL code 포함)	/product/{product_idx}
	
	<SQL code>
	//키워드 조회
	SELECT keyword FROM keyword WHERE product_idx = ?
	
	//추천 상품 조회
	SELECT DISTINCT
	sub.*,
	CASE
	WHEN ? IS NOT NULL AND EXISTS (
	SELECT 1
	FROM wishlist w
	WHERE w.product_idx = sub.product_idx AND w.user_idx = ?
	) THEN TRUE
	ELSE FALSE
	END AS is_hearted
	FROM (
	SELECT
	p.*,
	k.keyword
	FROM
	product p
	JOIN
	keyword k ON p.product_idx = k.product_idx
	WHERE
	k.keyword IN (%s)
	AND p.product_idx != ?
	AND p.status = 'active'
	) AS sub;

결과 화면	 카테고리 > 의류 남성 정장 세트 판매 50,000원 인천광역시 / 2024. 12. 11. 거래 중 / 구해요 ♡ 관심 6 · 채팅 3 작성 사이즈 100, 1회 착용. 삭제 키워드 기반 추천 상품 여성 겨울 코트 판매 40000원 수원시 ♡ 8 4  남성 티셔츠 판매 15000원 서울특별시 ♡ 9 5 

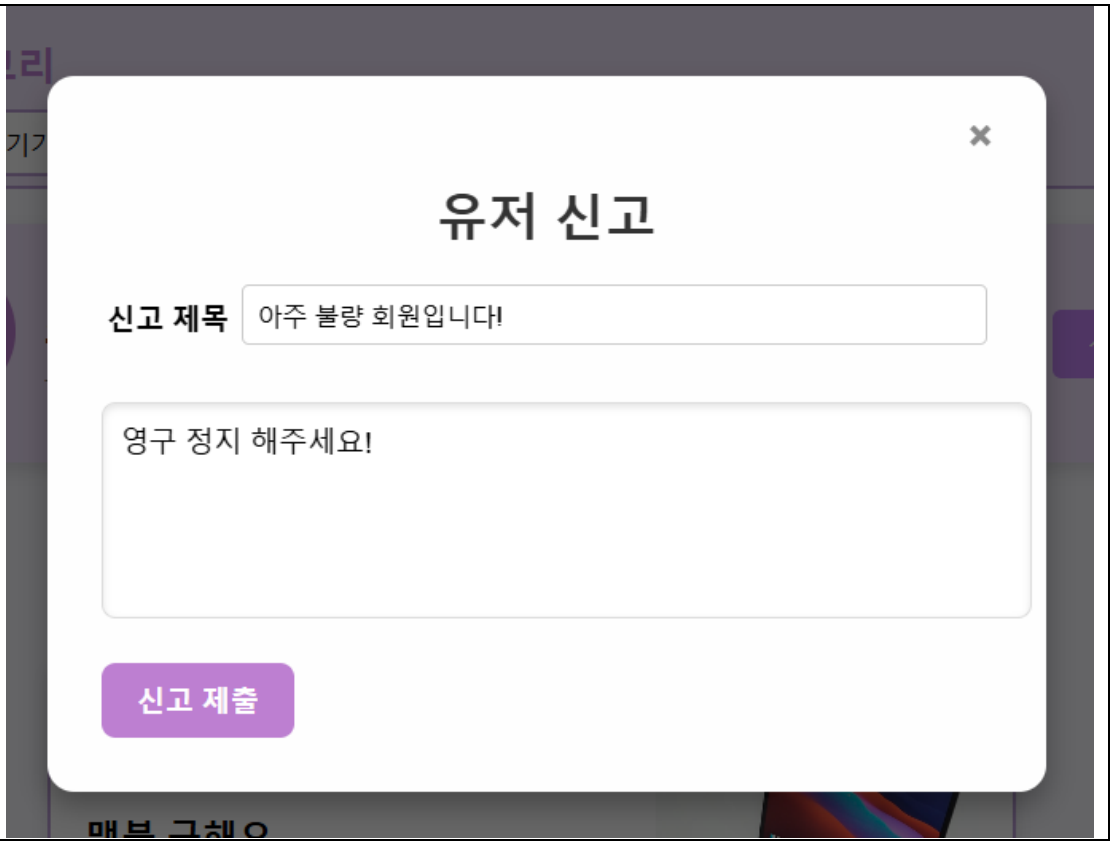
기능	상품 찜 등록/해제
기능 설명	사용자가 관심 있는 상품을 저장해둘 수 있는 기능을 제공합니다. 사용자는 마음에 드는 상품을 찜 목록에 추가하여 마이페이지에서 필요할 때 언제든지 쉽게 찾아볼 수 있습니다. 필요 없는 경우 찜 해제도 가능합니다.
기능 구현 방식 (API, SQL code 포함)	<API> /product/{product_idx}/wish, /product/{product_idx}/wish/cancel <SQL code> INSERT INTO Wishlist (user_idx, product_idx, created_at) VALUES (?, ?, ?) DELETE FROM Wishlist WHERE user_idx = ? AND product_idx = ?

결과 화면	 카테고리 > 기타 스포츠 용품 팝니다 50,000원 서울특별시 / 2024. 12. 9. 거래 중 / 구해요 ♡ 관심 4 · 채팅 1 작성자 소영이 여러 가지 스포츠 용품 판매 찜 해제 ♥ 채팅 신고

기능	상품 게시글 신고
기능 설명	사용자가 부적절한 게시물 또는 프로필을 발견했을 때 이를 신고할 수 있는 기능입니다. 신고된 내용은 관리자가 검토 후 적절한 조치를 취할 수 있습니다. 구체적인 신고 사유 작성란을 포함하여, 상세한 신고 정보를 수집할 수 있습니다. 등록된 신고는 Report 테이블에 저장되어 관리자가 확인합니다. 안전한 거래 환경을 보장하기 위한 기능입니다.
기능 구현 방식 (API, SQL code 포함)	<API> /report/product/{product_idx} <SQL code> -- 1. 상품 존재 확인 및 글쓴이 유저 인덱스 조회 SELECT writer_idx FROM Product WHERE product_idx = ?; -- 2. 신고 데이터 삽입 INSERT INTO Report (title, content, reporting_idx, status, reported_user, reported_product, created_at) VALUES (?, ?, ?, 'pending', ?, ?, NOW());

결과 화면	

기능	**회원 신고**
기능 설명	회원은 다른 회원의 프로필에서 해당 회원을 신고할 수 있습니다. 등록된 신고는 Report 테이블에 저장되어 관리자가 확인합니다.
기능 구현 방식 (API, SQL code 포함)	<API> /report/user/{user_idx} <SQL code> -- 1. 신고 대상 사용자 확인 SELECT COUNT(*) FROM user WHERE user_idx = ?; -- 2. 신고 내역 추가 INSERT INTO Report (title, content, reporting_idx, status, reported_user, created_at) VALUES (?, ?, ?, 'pending', ?, NOW());

결과 화면	

기능	신고 목록 조회
기능 설명	관리자는 신고 내역 페이지에서 신고 목록을 조회할 수 있습니다. Report 테이블에서 정보를 가져와 표 형태로 제목, 처리 상태, 신고 일자를 표시합니다. 각 신고 제목을 선택하면 상세 내용을 확인할 수 있습니다.
기능 구현 방식 (API, SQL code 포함)	<API> /admin/get/reportList <SQL code> SELECT report_idx, title, content, reporting_idx, status, reported_user, reported_product, created_at FROM report;
결과 화면	< 뒤로 가기 **신고 내역 관리** 순번 | 신고 제목 | 신고 처리 상태 | 신고 일자 1 | 상품 사기 신고 | 처리 전 | 2024. 11. 29. 2 | 욕설 및 불법 내용 포함 | 처리 완료 | 2024. 11. 28. 3 | 거래 조건 불이행 | 신고 취소 | 2024. 11. 27. 4 | 허위 광고 신고 | 처리 전 | 2024. 11. 26. 5 | 상품 이미지 도용 | 처리 완료 | 2024. 11. 25.

기능	신고 상세 조회

기능 설명	신고내역을 자세히 확인합니다. 신고 제목과 신고자, 신고일, 처리 상태, 신고 당한 회원, 신고하는 글, 신고 내용이 표시됩니다.
기능 구현 방식 (API, SQL code 포함)	<API> /report/{report_idx} <SQL code> SELECT r.report_idx, r.title, r.content, r.status, r.created_at, r.reporting_idx AS reporting_user_idx, reporting_user.id AS reporting_user_id, reporting_user.nickname AS reporting_user_nickname, r.reported_user AS reported_user_idx, reported_user.id AS reported_user_id, reported_user.nickname AS reported_user_nickname, r.reported_product AS reported_product_idx, p.title AS reported_product_title, p.content AS reported_product_content FROM Report r LEFT JOIN User reporting_user ON r.reporting_idx = reporting_user.user_idx LEFT JOIN User reported_user ON r.reported_user = reported_user.user_idx LEFT JOIN Product p ON r.reported_product = p.product_idx WHERE r.report_idx = ?;
결과 화면	< 뒤로 가기 허위 광고 신고 **신고자:** 은진이↗ **신고일:** 2024. 11. 26. **처리 상태:** 처리 전 **신고 당한 유저:** 길동이↗ **신고하는 글:** 채식주의자 구해요↗ **내용:** 해당 상품이 과장 광고로 판매되고 있습니다. 신고 반려 처리 완료

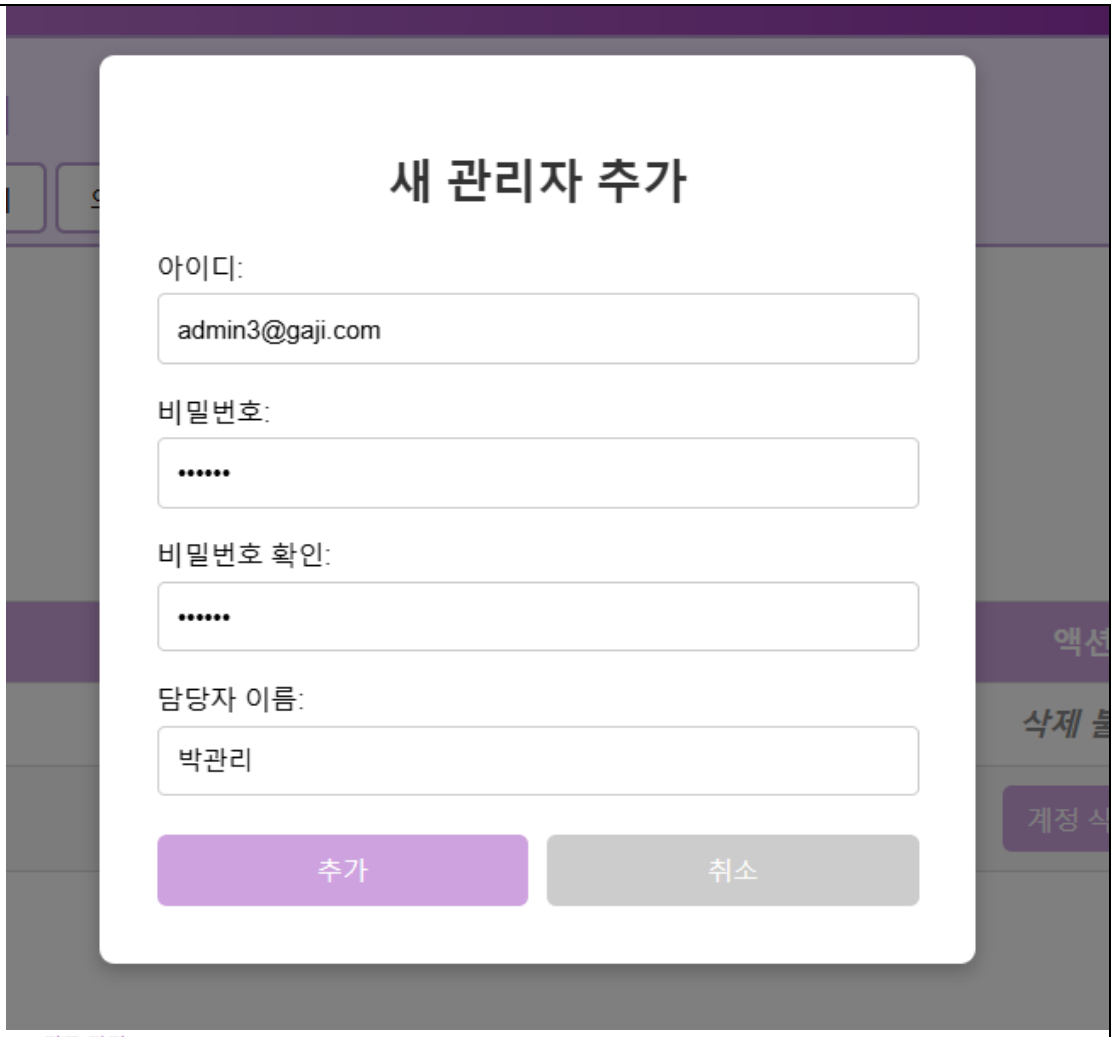
기능	신고 처리
기능 설명	관리자가 신고를 처리하는 방법은 '신고 반려'와 '처리 완료'가 있습니다. 신고 당한 회원이나 글에 문제가 없다고 판단한 경우에는 별도 처리 없이 신고를 반려하고, 회원 영구 정지나 상품 게시글 삭제 등 신고를 수용한 경우에는 완료 처리를 합니다. 두 기능 모두 Report 테이블에서 해당 신고 데이터를 찾아 상태를 업데이트합니다.

기능 구현 방식 (API, SQL code 포함)	<API> /report/{report_idx}/reject, /report/{report_idx}/resolve <SQL code> -- 1. 신고 존재 확인 SELECT status FROM Report WHERE report_idx = ?; -- 2. 신고 상태를 'rejected'로 변경 UPDATE Report SET status = 'rejected' WHERE report_idx = ?; -- 2. 신고 상태를 'resolved'로 변경 UPDATE Report SET status = 'resolved' WHERE report_idx = ?;
결과 화면	

기능	관리자 목록 조회
기능 설명	관리자는 관리자 페이지에서 관리자 계정 목록을 조회합니다. Admin 테이블에서 ID, 담당자 명을 가져옵니다.
기능 구현 방식 (API, SQL code 포	<API> /admin/get/adminList <SQL code> SELECT admin_index AS idx, id, name AS name FROM admin;

함)	
결과 화면	

기능	관리자 계정 추가
기능 설명	기존 관리자는 새로운 관리자를 추가하여 직원을 늘릴 수 있습니다. 아이디, 비밀번호, 담당자 이름을 입력하여 Admin 테이블에 등록합니다.
기능 구현 방식 (API, SQL code 포함)	<API> /admin/add <SQL code> -- 중복 검사 SELECT COUNT(*) FROM admin WHERE id = ?; -- 관리자 추가 INSERT INTO admin (id, passwd, name) VALUES (?, ?, ?);

결과 화면	 < 뒤로 가기 관리자 목록 관리자 추가 순번 | 아이디 | 담당자 | 액션 1 | admin1@gaji.com | 김관리 | 삭제 불가 2 | admin2@gaji.com | 나관리 | 계정 삭제 3 | admin3@gaji.com | 박관리 | 계정 삭제

기능	관리자 계정 삭제
기능 설명	관리자는 다른 관리자를 삭제할 수 있습니다. 이때 첫 번째 관리자는 기본 계정으로 삭제가 불가능합니다.
기능 구현 방식 (API, SQL code 포함)	<API> /admin/delete/{admin_idx} <SQL code>

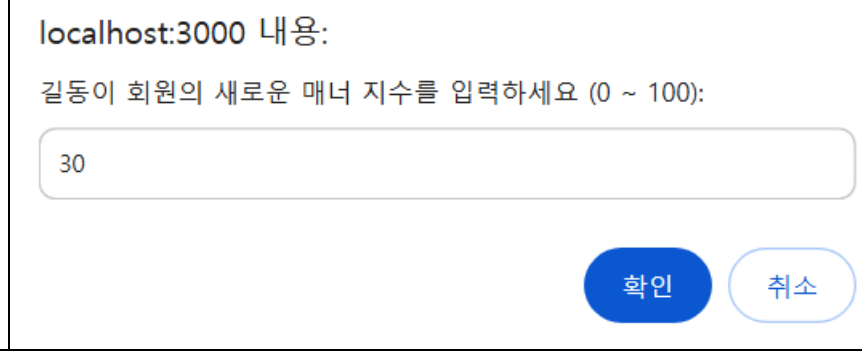
	-- 관리자 존재 여부 확인 SELECT COUNT(*) FROM admin WHERE admin_index = ?; -- 관리자 삭제 DELETE FROM admin WHERE admin_index = ?;
결과 화면	해당 기능 결과 화면은 따로 없습니다.

기능	**익명 문의**
기능 설명	회원이 타인에게 자신의 신원을 드러내지 않고 문의사항 페이지에서 관리자에게 문의할 수 있는 기능입니다. 이 기능을 통해 회원은 불안감 없이 관리자에게 플랫폼 사용법에 대한 질문을 하며, 이를 통해 더 많은 사용자 참여를 유도할 수 있습니다
기능 구현 방식 (API, SQL code 포함)	<API> /ask/write <SQL code> INSERT INTO Question (title, content, created_at, public, user_idx, image) VALUES (?, ?, NOW(), ?, ?, ?)
결과 화면	 문의한 회원과 관리자에게만 해당 문의 글이 보입니다.

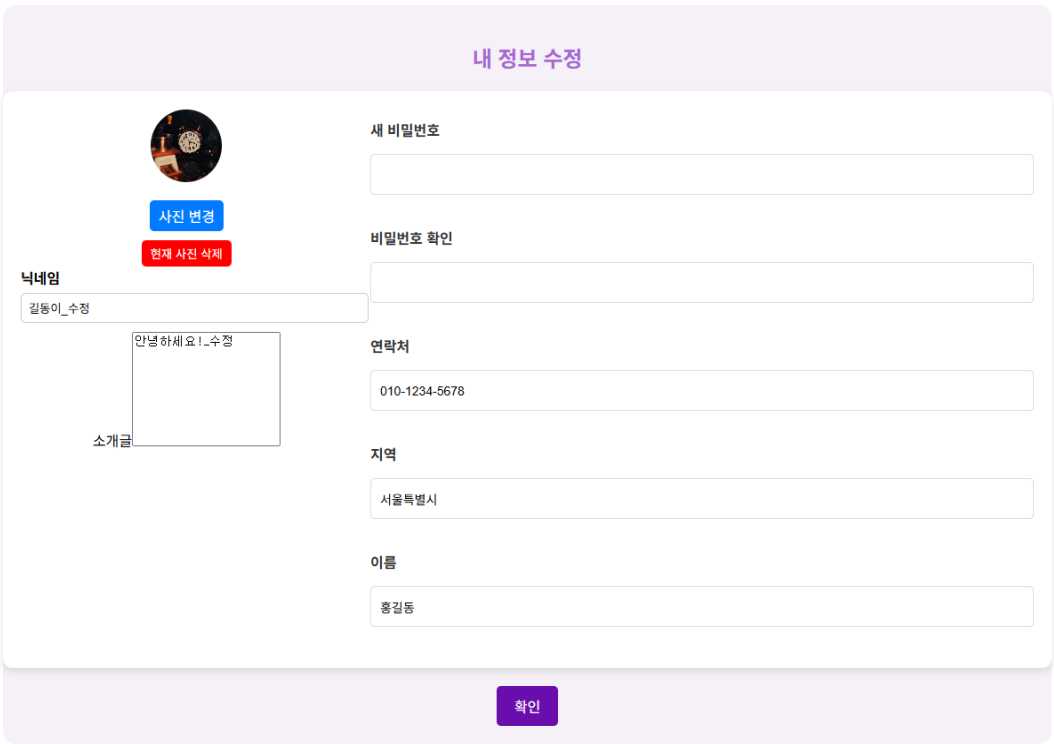
< 뒤로 가기

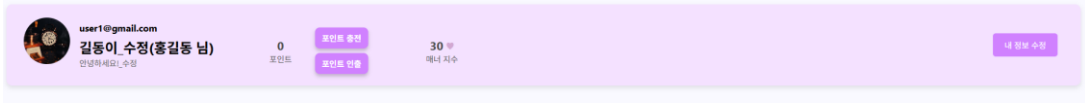
문의사항 게시판

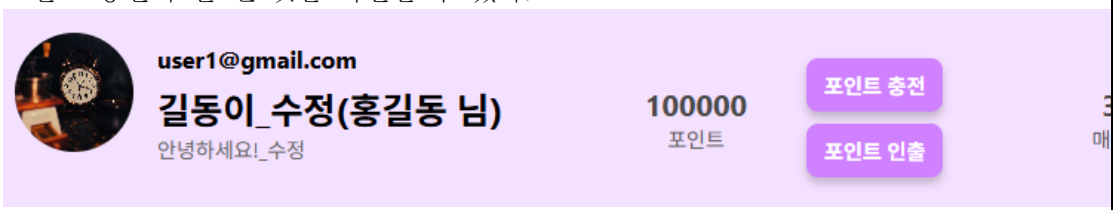
순번	제목	날짜	공개여부
6	비밀스럽게 문의하고 싶습니다.	2024. 12. 12.	비공개
5	다섯 번째 질문	2024. 12. 11.	공개
4	네 번째 질문	2024. 12. 10.	비공개
	ㄴ *Re: 네 번째 질문*	2024. 12. 12.	비공개
3	세 번째 질문	2024. 12. 9.	공개
2	두 번째 질문	2024. 12. 7.	비공개
1	첫 번째 질문	2024. 12. 5.	공개
	ㄴ *Re: 첫 번째 질문*	2024. 12. 12.	공개

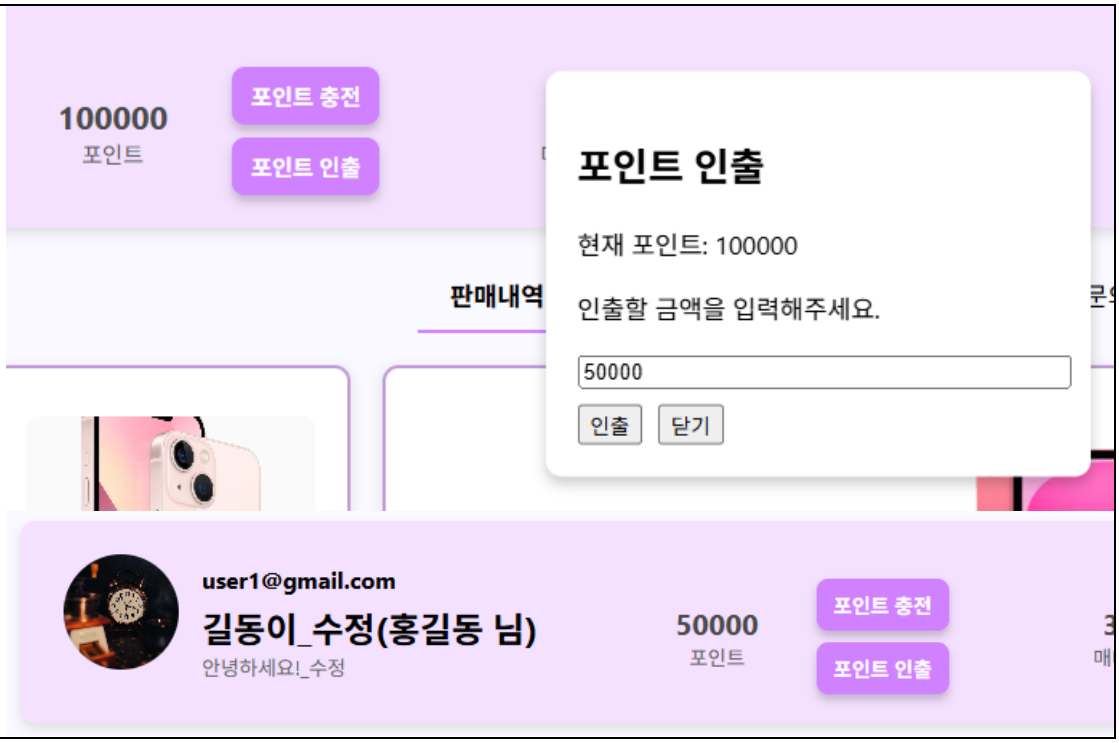
기능	**관리자가 매너 지수 설정**
기능 설명	이 기능은 관리자가 특정 사용자의 매너 지수를 수정할 수 있도록 합니다. 관리자는 사용자 식별자(user_idx)와 새로운 매너 지수를 입력하여 해당 사용자의 매너 점수를 업데이트할 수 있습니다. 매너 지수는 사용자의 신뢰도와 플랫폼 내 행동 평가를 나타내며, 관리자가 필요에 따라 이를 조정할 수 있는 권한을 가집니다. 이는 플랫폼 커뮤니티의 질서를 유지하고 공정한 환경을 조성하는 데 기여합니다.
기능 구현 방식 (API, SQL code 포함)	<API> /user/{user_idx}/mannerpoint/update <SQL code> UPDATE user SET manner_point = ? WHERE user_idx = ?
결과 화면	

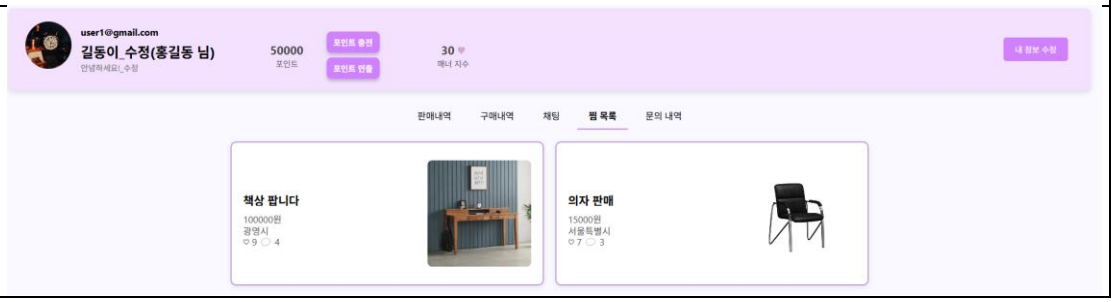
	 길동이의 매너 지수가 30으로 변경된 것을 확인할 수 있다.

기능	내 정보 수정(프로필 사진 및 소개글 설정 포함)
기능 설명	이 기능은 사용자가 자신의 정보를 수정할 수 있도록 제공합니다. 사용자는 이름, 전화번호, 지역, 비밀번호, 프로필 사진, 소개글, 닉네임을 변경할 수 있습니다. 또한, 닉네임이 변경될 경우 이를 반영하기 위해 상품 데이터에서도 작성자 이름(writer_name)이 업데이트됩니다. 이를 통해 사용자 정보의 일관성을 유지하고, 변경된 정보가 즉시 반영되도록 합니다.
기능 구현 방식 (API, SQL code 포함)	<API> /user/{email}/edit <SQL code> 1. 사용자 정보 수정 UPDATE user SET name = ?, phone = ?, location = ?, passwd = ?, image = ?, message = ?, nickname = ? WHERE id = ? 2. 상품 작성자 정보 수정 UPDATE product SET writer_name = ? WHERE writer_name = ?
결과 화면	 수정이 원하는 대로 반영된 것을 확인할 수 있다.

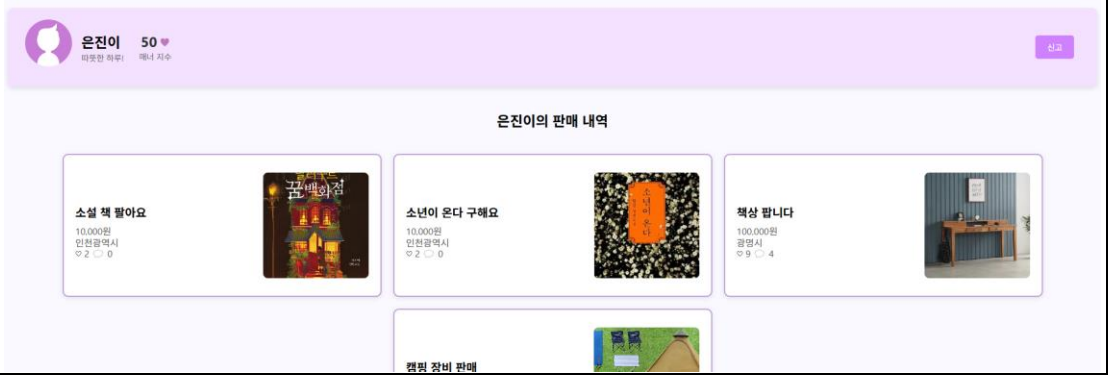
	

기능	포인트 충전 및 인출 기능
기능 설명	포인트 충전 및 인출 기능은 사용자가 자신의 계정에서 포인트를 충전하거나 인출할 수 있도록 합니다. 사용자는 포인트를 충전 또는 인출하려는 금액을 입력하며, 입력된 금액은 point 테이블에 업데이트됩니다. 동시에, 포인트 변동 내역은 Point_history 테이블에 기록되어 사용자와 관리자가 트랜잭션 기록을 추적할 수 있도록 합니다. 충전과 인출 모두 동일한 프로세스를 사용하며, 금액이 음수일 경우 인출로 처리됩니다.
기능 구현 방식 (API, SQL code 포함)	<API> /user/{email}/point/update <SQL code> 1. 포인트 업데이트 INSERT INTO point (user_idx, point) VALUES ((SELECT user_idx FROM user WHERE id = ?), ?) ON DUPLICATE KEY UPDATE point = point + ? 2. Point_history 테이블에 기록 INSERT INTO Point_history (user_idx, change_amount, transaction_type, created_at) VALUES ((SELECT user_idx FROM user WHERE id = ?), ?, ?, NOW())
결과 화면	 포인트 충전이 잘 된 것을 확인할 수 있다.  인출도 마찬가지로 잘 동작한다.

기능	찜 목록 가져오기
기능 설명	찜 목록 가져오기 기능은 사용자가 자신이 관심 상품으로 저장한 항목들을 확인할 수 있도록 합니다. 사용자는 프로필 페이지에서 본인이 찜한 상품들을 조회할 수 있으며, 상품 정보에는 제목, 가격, 위치, 좋아요 수, 채팅 수, 대표 이미지 등이 포함됩니다. 이 기능을 통해 사용자는 마음에 드는 상품을 빠르게 다시 확인하고 필요에 따라 상품 상세 페이지로 이동할 수 있습니다.
기능 구현 방식 (API, SQL code 포함)	<API> /user/{email}/get/wishlist <SQL code> SELECT p.product_idx, p.title, p.price, p.location, p.heart_num, p.chat_num, p.image FROM Product p RIGHT JOIN Wishlist w ON p.product_idx = w.product_idx WHERE w.user_idx = (SELECT user_idx FROM User WHERE id = ?)
결과 화면	

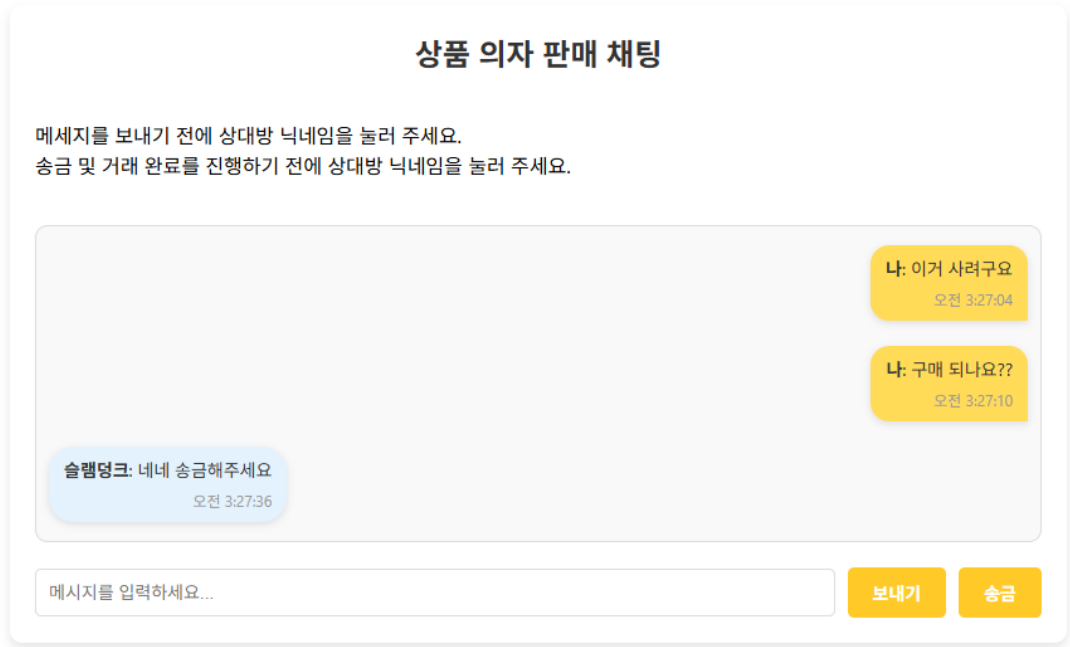
기능	판매자 프로필 가져오기
기능 설명	판매자 프로필 가져오기 기능은 특정 판매자의 프로필 정보를 조회할 수 있도록 합니다. 이 기능은 사용자가 상품 상세 페이지 등에서 해당 상품의 판매자 정보를 확인하려 할 때 사용됩니다. 판매자의 닉네임, 소개글, 매너 점수, 프로필 이미지와 같은 정보가 제공되어 신뢰도를 판단하거나 판매자와 소통할 수 있는 근거를 제공합니다.

기능 구현 방식 (API, SQL code 포함)	<API> /user/author/{user_idx} <SQL code> 1. 판매자 정보 가져오기 SELECT user_idx, id, name, nickname, message, manner_point, image FROM user WHERE user_idx = ? 2. 판매자가 판매하는 상품 가져오기 SELECT product_idx, title, price, location, heart_num, chat_num, image, status FROM product WHERE writer_idx = ?
결과 화면	상품 상세 페이지의 판매자 닉네임을 클릭하면 판매자의 프로필로 이동됩니다. 동시에, 판매자가 판매하는 다른 상품들도 확인할 수 있습니다. 

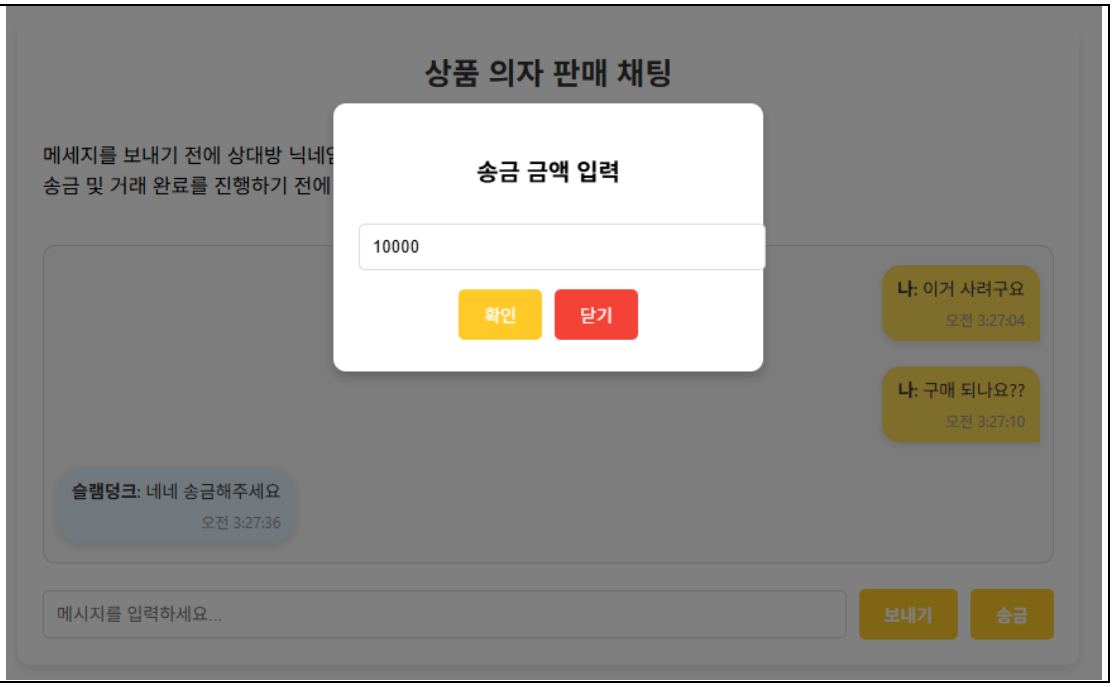
기능	**마이페이지에서 구매한 상품 내역 조회**
기능 설명	구매한 상품 내역 조회 기능은 사용자가 자신의 프로필 페이지에서 과거에 구매한 상품의 목록을 확인할 수 있도록 합니다. 이 기능은 사용자가 거래 내역을 추적하거나 과거 구매한 상품을 다시 확인하는 데 유용합니다. 데이터베이스에서 구매자의 식별자(partner_idx)를 기준으로 상품 정보를 가져옵니다.
기능 구현 방식 (API, SQL code 포함)	<API> /product/purchases <SQL code> SELECT * FROM Product WHERE partner_idx = ?
결과 화면	

기능	**채팅 기능**
기능 설명	채팅 기능은 사용자가 특정 상품과 관련하여 다른 사용자(판매자 또는 구매자)와 소통할 수 있도록 합니다. 사용자는 실시간으로 메시지를 주고받으며, 대화 내용은 데이터베이

	스에 저장됩니다. 또한, 마이페이지에서 해당 상품에 대한 이전 채팅 기록을 확인할 수 있습니다.
기능 구현 방식 (API, SQL code 포함)	<API> /chat/send, /chat/get/chatList, /chat/{product_idx} <SQL code> 1. 채팅 메시지 저장 INSERT INTO Chat (sender_idx, receiver_idx, content, created_at, product_idx) VALUES (?, ?, ?, NOW(), ?) 2. 특정 상품의 채팅 기록 가져오기 SELECT c.*, u.nickname AS senderNickname, p.title AS productTitle FROM Chat c JOIN User u ON c.sender_idx = u.user_idx JOIN Product p ON c.product_idx = p.product_idx WHERE c.product_idx = ? ORDER BY c.created_at ASC 3. 사용자의 모든 채팅 목록 가져오기 SELECT c.*, u.nickname AS senderNickname, p.title AS productTitle FROM Chat c JOIN User u ON c.sender_idx = u.user_idx JOIN Product p ON c.product_idx = p.product_idx WHERE c.sender_idx = ? OR c.receiver_idx = ? ORDER BY c.created_at ASC
결과 화면	마이페이지의 채팅 페이지에서 기존 채팅 목록 확인 가능합니다.  특정 상품 채팅을 클릭하면 채팅 기록이 나타납니다.

기능	채팅 중 송금 기능
기능 설명	채팅 중 송금 기능은 사용자가 채팅을 통해 거래를 진행하며 포인트를 송금할 수 있도록 합니다. 이 기능은 구매자가 판매자에게 포인트를 전송하는 방식으로 작동하며, 사용자의 보유 포인트를 확인하고, 송금 후 업데이트하며, 송금 기록을 저장합니다. 이를 통해 거래의 투명성과 안전성을 보장합니다.
기능 구현 방식 (API, SQL code 포함)	<API> /chat/transaction/transfer <SQL code> 1. 사용자 포인트 조회 SELECT point FROM Point WHERE user_idx = ? 2. 사용자 포인트 업데이트 UPDATE Point SET point = ? WHERE user_idx = ? 3. 포인트 변동 기록 저장 INSERT INTO Point_history (user_idx, change_amount, transaction_type, created_at) VALUES (?, ?, ?, NOW())

결과 화면	

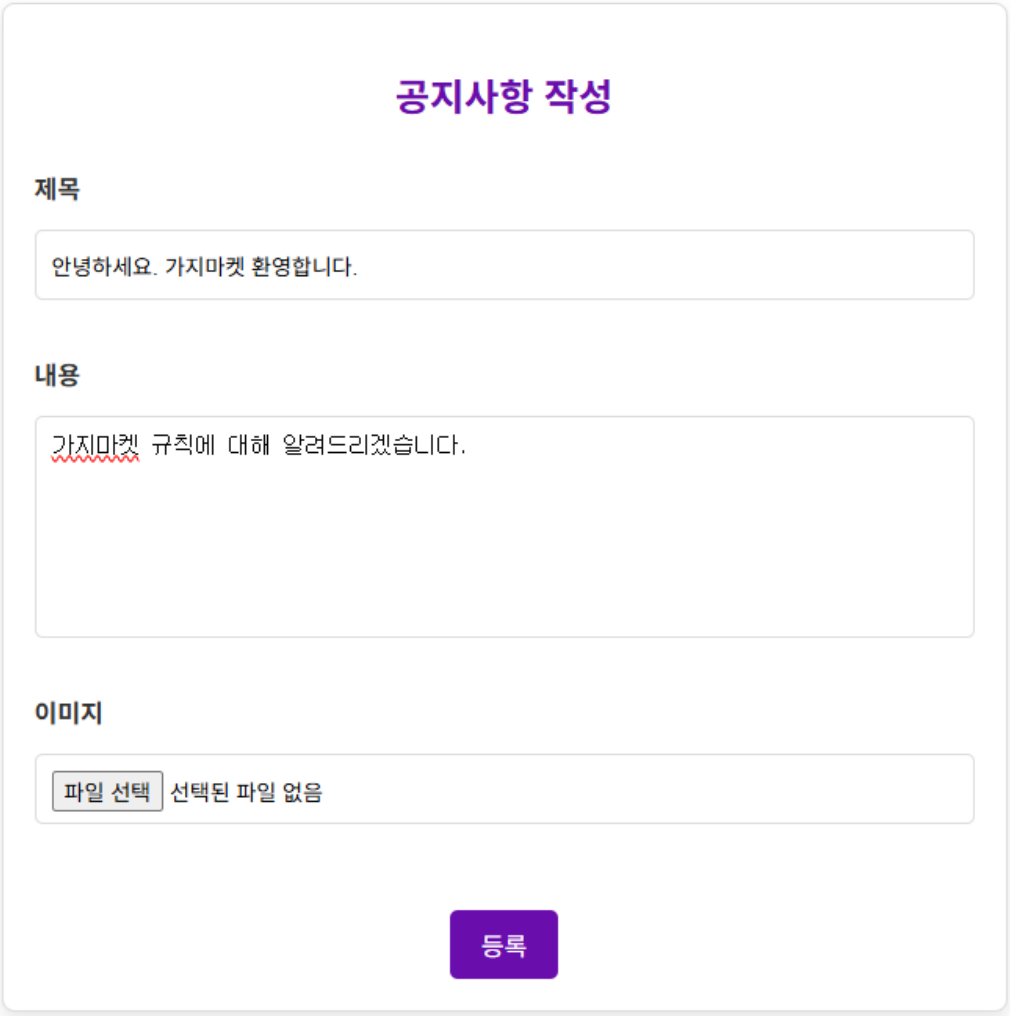
기능	공지 목록 가져오기
기능 설명	공지 목록 가져오기 기능은 사용자가 공지사항 페이지에서 게시된 모든 공지사항을 확인할 수 있도록 합니다. 데이터베이스에서 공지사항 정보를 가져오며, 관리자 정보도 함께 조회됩니다. 각 공지사항은 작성자 이름, 제목, 내용, 작성일, 이미지 등의 정보를 포함합니다. 공지사항은 고유 식별자(notice_idx)를 기준으로 정렬되어 표시됩니다.
기능 구현 방식 (API, SQL code 포함)	<API> /notice, /notice/{notice_idx} <SQL code> 1. 공지 목록 가져오기 SELECT n.notice_idx AS noticeId, n.admin_index AS adminId, n.title AS noticeTitle, n.content AS noticeContent, n.created_at AS noticeCreatedAt, n.image AS noticeImage, a.name AS adminName FROM Notice n JOIN Admin a ON n.admin_index = a.admin_index ORDER BY n.notice_idx ASC 2. 공지 상세 조회 SELECT n.notice_idx AS noticeId, n.admin_index AS adminId, n.title AS noticeTitle, n.content AS noticeContent, n.created_at AS noticeCreatedAt, n.image AS noticeImage, a.name AS adminName FROM Notice n JOIN Admin a ON n.admin_index = a.admin_index WHERE n.notice_idx = ?
결과 화면	공지사항 목록이 잘 출력되는 것을 확인할 수 있다.

<table>
<tr>
<td></td>
<td>

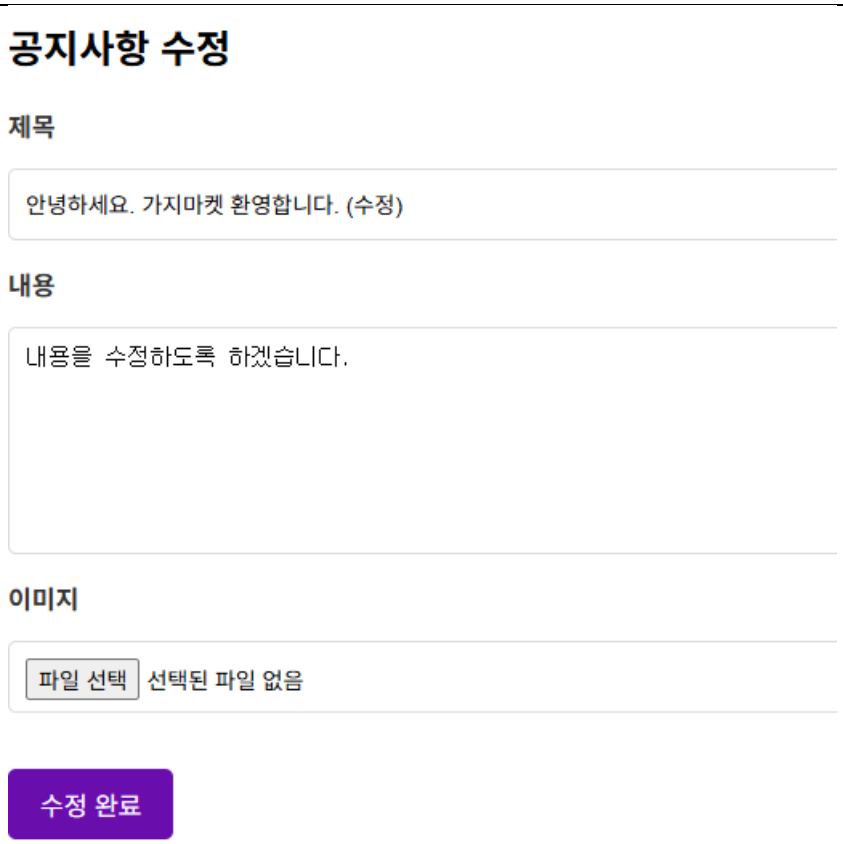

원하는 공지를 클릭하면 해당 공지의 상세 조회가 가능한 것을 확인할 수 있다.


</td>
</tr>
</table>

기능	**관리자가 공지 작성**
기능 설명	관리자가 공지 작성 기능은 플랫폼의 관리자(Admin)가 사용자들에게 전달할 공지사항을 생성할 수 있도록 합니다. 공지사항에는 제목, 내용, 작성 시간, 관련 이미지가 포함됩니다. 작성된 공지는 공지 목록에서 사용자들이 확인할 수 있습니다.
기능 구현 방식 (API, SQL code 포함)	<API> /notice/upload <SQL code> INSERT INTO Notice (admin_index, title, content, created_at, image) VALUES (?, ?, ?, NOW(), ?)

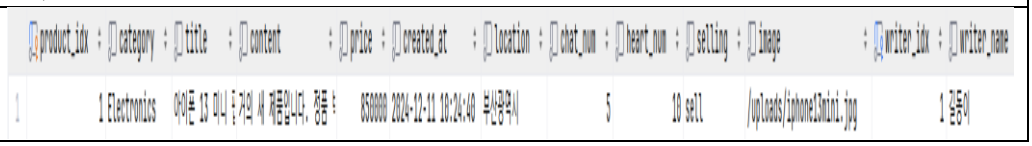
결과 화면	 공지사항이 잘 작성된 것을 확인할 수 있다. 

기능	관리자가 공지 수정 및 삭제
기능 설명	관리자가 공지 수정 및 삭제 기능을 통해 기존 공지사항을 편집하거나 제거할 수 있습니다. 수정 기능: 공지사항의 제목, 내용, 첨부 이미지를 변경합니다. 삭제 기능: 공지사항을 데이터베이스에서 제거합니다. 이 기능은 관리자의 공지 관리 편의성을 높이고, 필요에 따라 공지사항을 최신 상태로 유지하거나 불필요한 공지사항을 삭제할 수 있도록 합니다. 공지 수정 및 삭제는 관리자의 권한이므로, 일반 사용자는 수정 및 삭제 버튼이 아예 보이지 않습니다.
기능 구현 방식 (API,	<API> /notice/delete/{notice_idx}, /notice/edit/{notice_idx} <SQL code> 1. 공지 수정

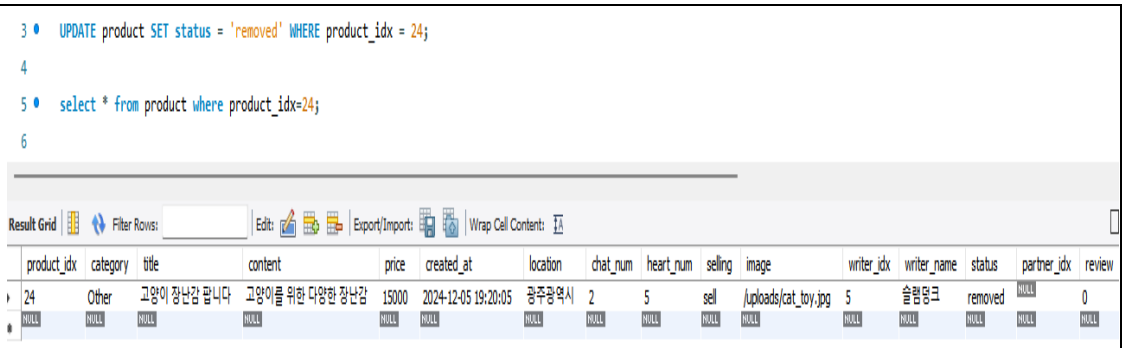
SQL code 포함)	UPDATE Notice SET title = ?, content = ?, image = ? WHERE notice_idx = ? 2. 공지 삭제 DELETE FROM Notice WHERE notice_idx = ?
결과 화면	 수정이 잘 반영된 것을 확인할 수 있다.  이후에, 삭제 버튼을 누르면 해당 공지사항이 잘 삭제되는 것을 확인할 수 있다. 

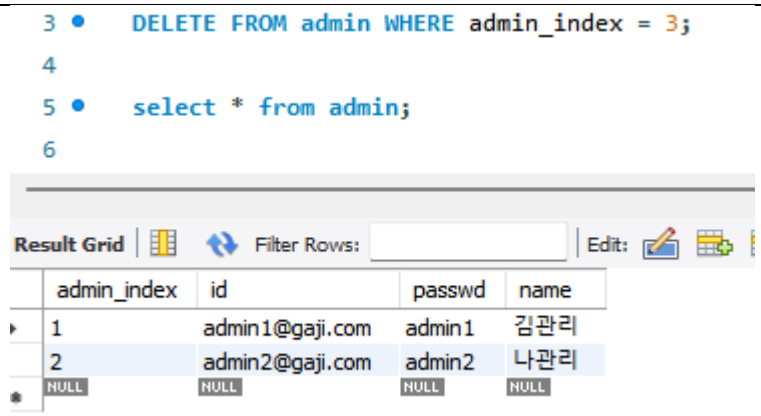
3. SQL 설명

SQL 설명은 각 SQL 종류별로 각 대표 예시 하나씩 만 선정하여 작성하였습니다.

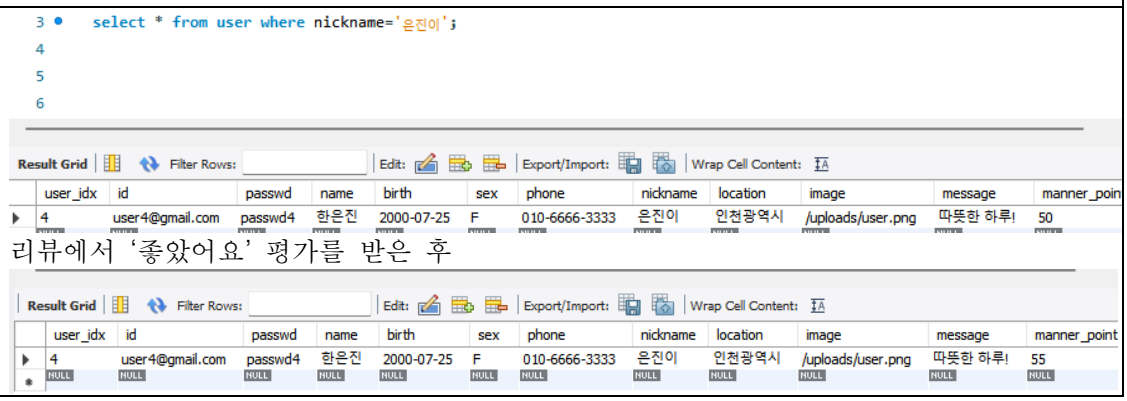
SQL 종류	INSERT
SQL 사용 위치	상품 등록
SQL 사용 설명	INSERT 쿼리를 이용해 product 테이블에 상품 정보를 등록합니다. 인덱스 번호에 auto_increment를 설정하여 따로 수를 입력하지 않고, 자동으로 증가가 됩니다.
사용한 SQL 문	INSERT INTO product (category, title, content, price, created_at, location, chat_num, heart_num, selling, image, writer_idx, writer_name, status) VALUES ('Electronics', '아이폰 13 미니 팝니다', '거의 새 제품입니다. 정품 박스 포함.', 850000, NOW(), '부산광역시', 5, 10, 'sell', '/uploads/iphone13mini.jpg', 1, '길동이', 'active');
데이터베이스 실행 화면	

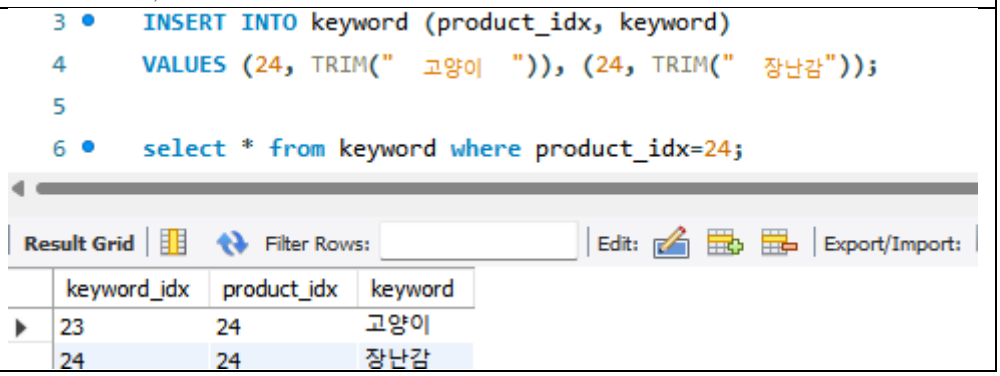
SQL 종류	UPDATE
SQL 사용 위치	상품 삭제
SQL 사용 설명	회원 또는 관리자가 상품을 삭제한 경우, 실제로 상품 글은 삭제되지 않고 '삭제' 상태로 업데이트 됩니다.
사용한 SQL 문	UPDATE product SET status = 'removed' WHERE product_idx = ?

데이터베이스 실행 화면	

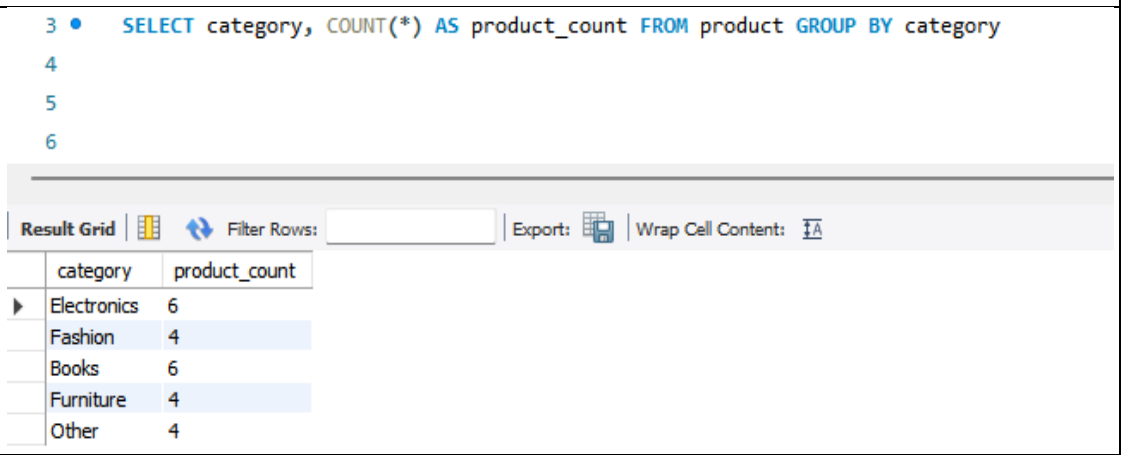
SQL 종류	DELETE
SQL 사용 위치	관리자 계정 삭제
SQL 사용 설명	관리자 계정을 admin 테이블에서 삭제하여 해당 계정이 더 이상 관리자로서 플랫폼에 접근할 수 없게 합니다.
사용한 SQL 문	DELETE FROM admin WHERE admin_index = ?;
데이터베이스 실행 화면	

SQL 종류	CASE
SQL 사용 위치	상품 후기 작성
SQL 사용 설명	리뷰에서 '좋았어요' 평가를 받은 경우 상품 게시글 작성자의 매너 지수가 5 상승하고, '아쉬웠어요' 평가를 받은 경우 5 하강하는 것을 CASE문을 이용하여 구현합니다.
사용한	UPDATE user SET manner_point = manner_point + CASE

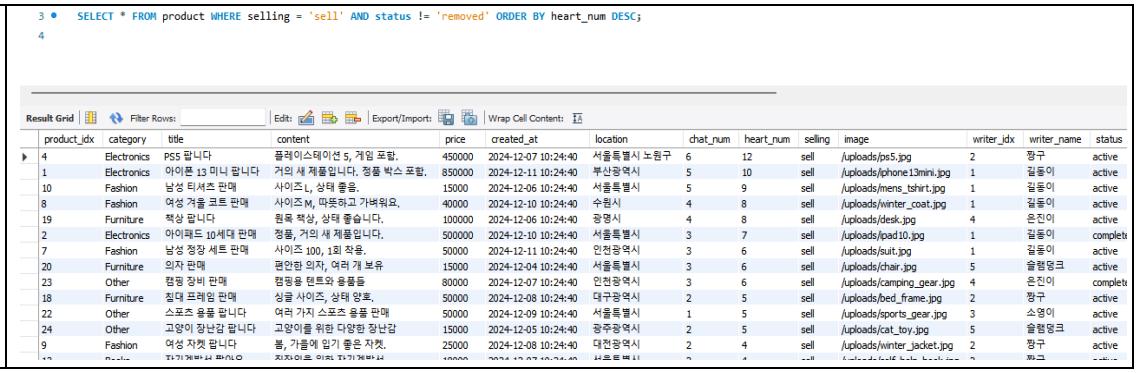
SQL 문	WHEN ? = 'good' THEN 5 WHEN ? = 'bad' THEN -5 ELSE 0 END WHERE user_idx = ?
데이터베이스 실행 화면	 리뷰에서 '좋았어요' 평가를 받은 후

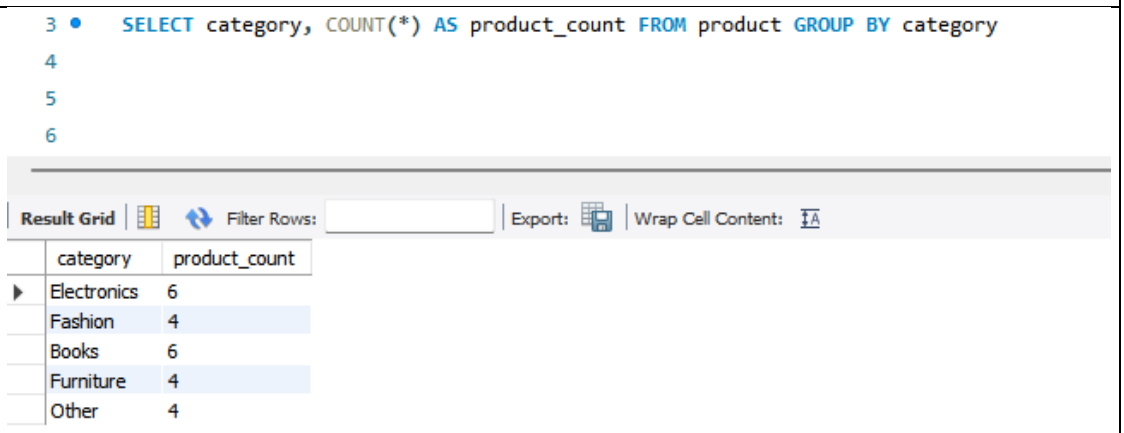
SQL 종류	TRIM
SQL 사용 위치	상품 게시글 작성
SQL 사용 설명	상품 게시글 등록 시 회원은 상품에 관한 키워드를 함께 등록합니다. [" 고양이", " 장난감 ", " 동물 "]처럼 공백을 함께 입력했을 때 앞뒤 공백을 제거 후 keyword 테이블에 저장합니다.
사용한 SQL 문	INSERT INTO keyword (product_idx, keyword) VALUES (?, TRIM(?))
데이터베이스 실행 화면	

SQL 종류	COUNT
SQL 사용위치	카테고리별 상품 수 데이터 시각화
SQL 사용	카테고리별 상품 수를 알기 위해 category를 기준으로 그룹을 묶고, product 테이블에서 해당 카테고리에 속하는 데이터들의 개수를 셉니다.

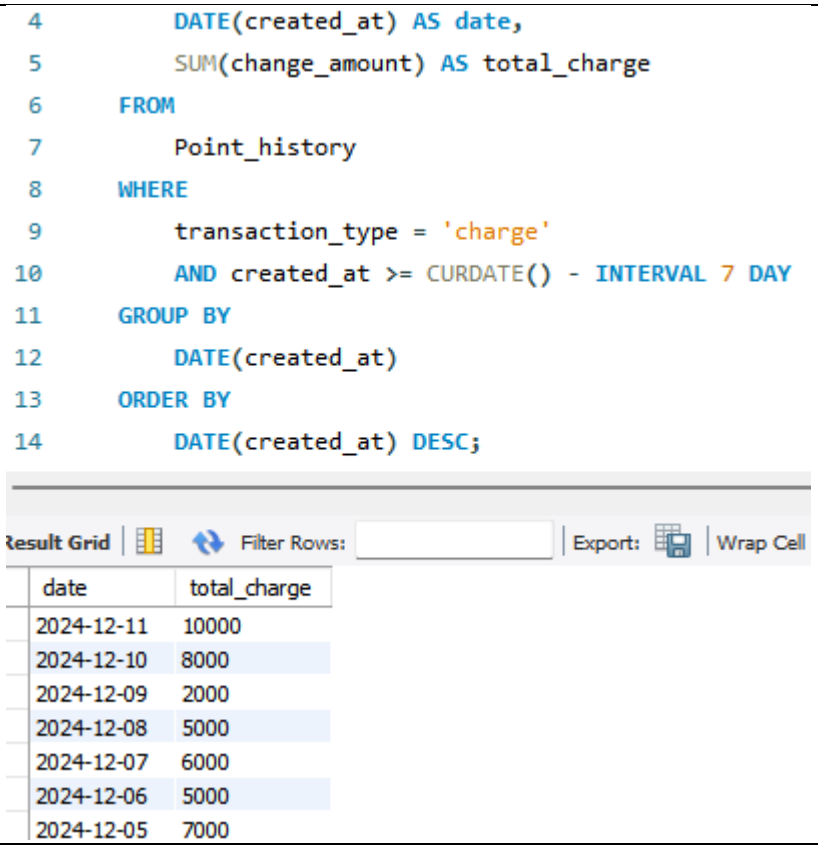
설명	
사용한 SQL 문	SELECT category, COUNT(*) AS product_count FROM product GROUP BY category
데이터베이스 실행화면	

SQL 종류	ORDER BY
SQL 사용위치	상품 전체 조회
SQL 사용설명	모든 상품 목록에서 삭제되지 않은 상품 중 '팔아요'인 상품을 인기순(찜이 많은 순)으로 정렬하여 가져오는 모습입니다.
사용한 SQL 문	SELECT * FROM product WHERE selling = 'sell' AND status != 'removed' ORDER BY heart_num DESC

데이터베이스 실행 화면	

SQL 종류	GROUP BY
SQL 사용 위치	카테고리별 상품 수 데이터 시각화
SQL 사용 설명	카테고리별 상품 수를 시각화하는 기능에서 카테고리를 기준으로 데이터를 그룹화하여 COUNT합니다. 그룹화할 때 GROUP BY를 사용합니다.
사용한 SQL 문	SELECT category, COUNT(*) AS product_count FROM product GROUP BY category
데이터베이스 실행 화면	

SQL 종류	SUM
SQL 사용 위치	최신 일주일 포인트 충전량 데이터 시각화
SQL 사용 설	일주일 동안 플랫폼 시스템에서 충전된 총 포인트량을 구하는 기능으로,

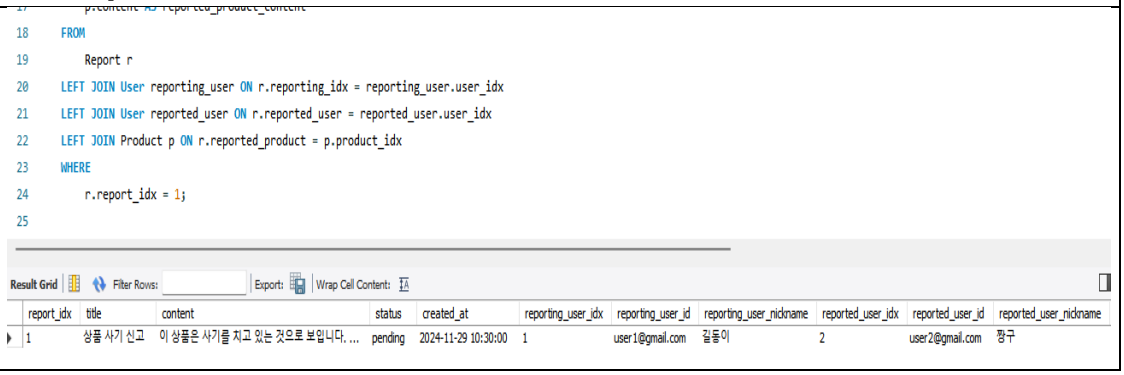
명	point_history 테이블에서 충전(charge)인 데이터의 포인트량을 모두 더합니다.
사용한 SQL 문	SELECT DATE(created_at) AS date, SUM(change_amount) AS total_charge FROM Point_history WHERE transaction_type = 'charge' AND created_at >= CURDATE() - INTERVAL 7 DAY GROUP BY DATE(created_at) ORDER BY DATE(created_at) DESC;
데이터베이스 실행 화면	

```
4          DATE(created_at) AS date,
5          SUM(change_amount) AS total_charge
6      FROM
7          Point_history
8      WHERE
9          transaction_type = 'charge'
10         AND created_at >= CURDATE() - INTERVAL 7 DAY
11     GROUP BY
12         DATE(created_at)
13     ORDER BY
14         DATE(created_at) DESC;
```

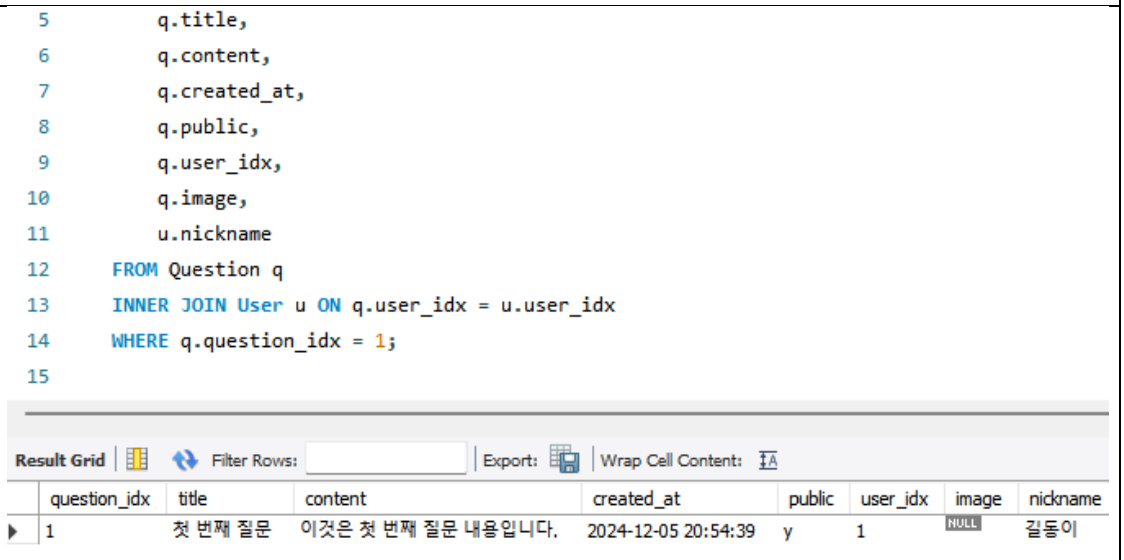
Result Grid | Filter Rows: | Export: | Wrap Cell

date	total_charge
2024-12-11	10000
2024-12-10	8000
2024-12-09	2000
2024-12-08	5000
2024-12-07	6000
2024-12-06	5000
2024-12-05	7000

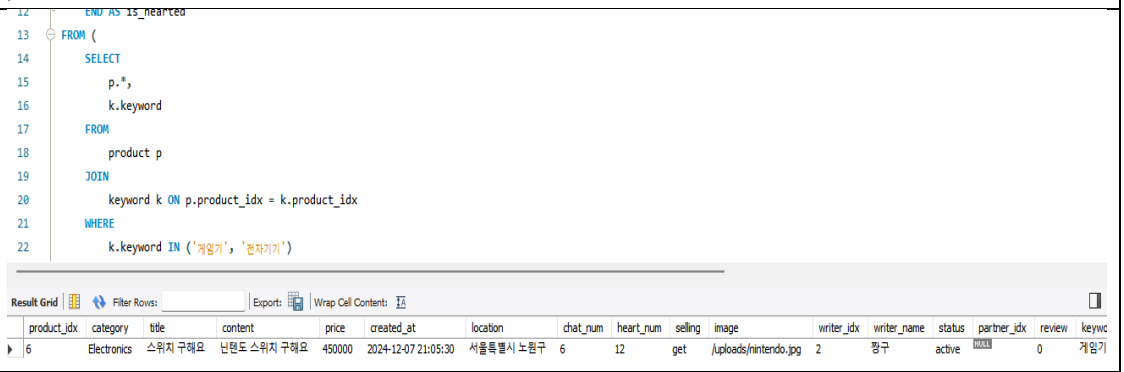
SQL 종류	LEFT JOIN
SQL 사용 위치	신고 상세 조회
SQL	신고글을 상세 조회할 때 쓰는 SQL문으로, report 테이블에는 회원 닉네임, 상품 글 제목과 같은 부가적인 정보는 기록이 안 되어 있고 product, user 테이블과 인덱스 키로

사용설명	연결되어 있습니다. 신고와 관련된 회원과 상품 글의 닉네임, 제목, 내용 등 필요한 정보를 가져옵니다.
사용한SQL문	SELECT r.report_idx, r.title, r.content, r.status, r.created_at, r.reporting_idx AS reporting_user_idx, reporting_user.id AS reporting_user_id, reporting_user.nickname AS reporting_user_nickname, r.reported_user AS reported_user_idx, reported_user.id AS reported_user_id, reported_user.nickname AS reported_user_nickname, r.reported_product AS reported_product_idx, p.title AS reported_product_title, p.content AS reported_product_content FROM Report r LEFT JOIN User reporting_user ON r.reporting_idx = reporting_user.user_idx LEFT JOIN User reported_user ON r.reported_user = reported_user.user_idx LEFT JOIN Product p ON r.reported_product = p.product_idx WHERE r.report_idx = ?;
데이터베이스실행화면	

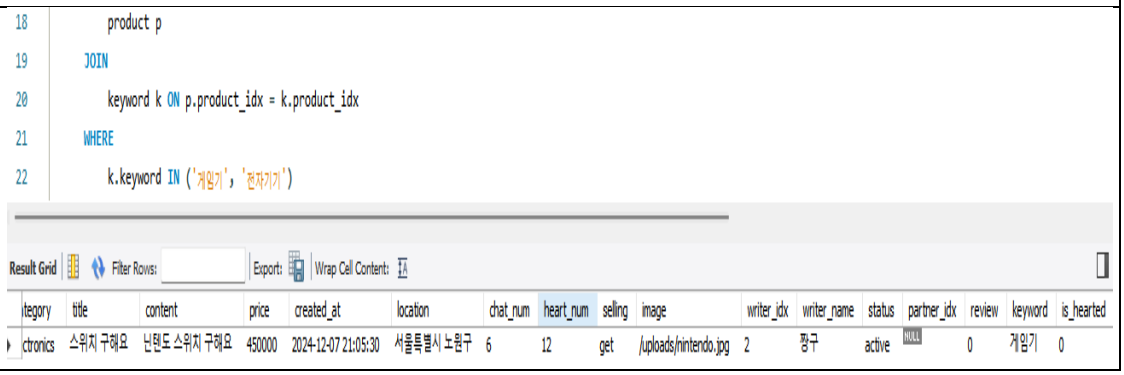
SQL종류	INNER JOIN
SQL사용위치	문의글 조회
SQL사	문의글을 저장한 question 테이블에는 회원 정보가 user 테이블의 Primary key인 인덱스로 저장되어 있습니다. 이를 이용하여 INNER JOIN하여 회원의 닉네임을 가져옵니다.

용설명	
사용한SQL문	SELECT q.question_idx, q.title, q.content, q.created_at, q.public, q.user_idx, q.image, u.nickname FROM Question q INNER JOIN User u ON q.user_idx = u.user_idx WHERE q.question_idx = ?
데이터베이스실행화면	

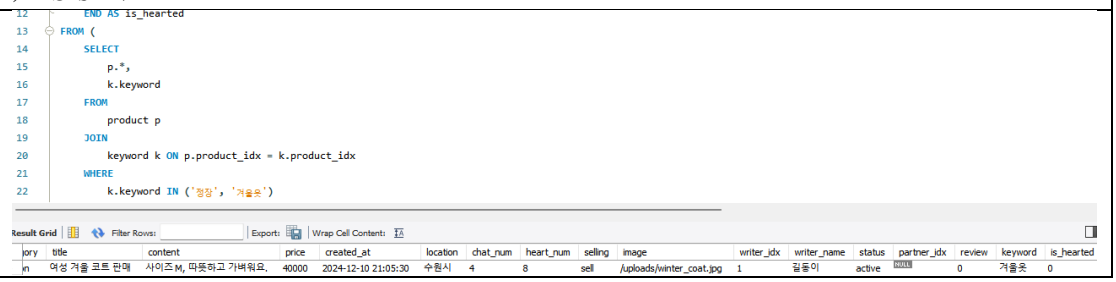
SQL종류	Subqueries in the from clause
SQL사용위치	상품 개별 조회
SQL사용설명	상품 게시글 조회 시, 키워드가 일치하는 추천 상품을 함께 제시해야 합니다. 서브쿼리에서 product 테이블과 keyword 테이블을 조인하고, 특정 조건에 맞는 추천 상품을 필터링하여 데이터를 반환합니다. 이 서브쿼리의 결과는 sub으로 저장 되어 이후 외부 쿼리에서 활용됩니다.
사용	3. 추천 상품 조회 SELECT DISTINCT

한 SQL 문	sub.*, CASE WHEN ? IS NOT NULL AND EXISTS (SELECT 1 FROM wishlist w WHERE w.product_idx = sub.product_idx AND w.user_idx = ?) THEN TRUE ELSE FALSE END AS is_hearted FROM (SELECT p.*, k.keyword FROM product p JOIN keyword k ON p.product_idx = k.product_idx WHERE k.keyword IN (%s) AND p.product_idx != ? AND p.status = 'active') AS sub;
데이터베이스 실행화면	

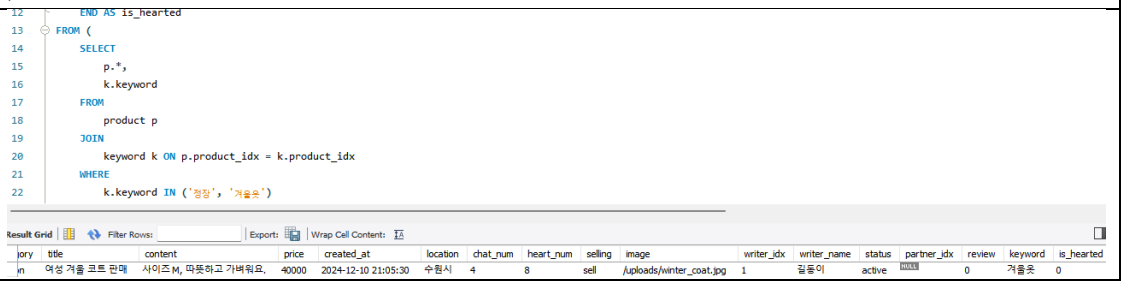
SQL 종류	EXISTS
SQL 사용위치	상품 개별 조회
SQL 사용설명	상품 게시글 조회 시 키워드 기반 추천 상품 정보를 가져올 때 현재 조회하는 상품이 현재 사용자의 위시 리스트(찜 목록)에 있는지 확인하기 위해 EXISTS를 씁니다. 찜한 상품이면 is_hearted가 true(1)이고 찜하지 않은 상품이면 false(0)입니다.
사	SELECT DISTINCT

용한 SQL 문	sub.*, CASE WHEN ? IS NOT NULL AND EXISTS (SELECT 1 FROM wishlist w WHERE w.product_idx = sub.product_idx AND w.user_idx = ?) THEN TRUE ELSE FALSE END AS is_hearted FROM (SELECT p.*, k.keyword FROM product p JOIN keyword k ON p.product_idx = k.product_idx WHERE k.keyword IN (%s) AND p.product_idx != ? AND p.status = 'active') AS sub;
데이터베이스 실행화면	

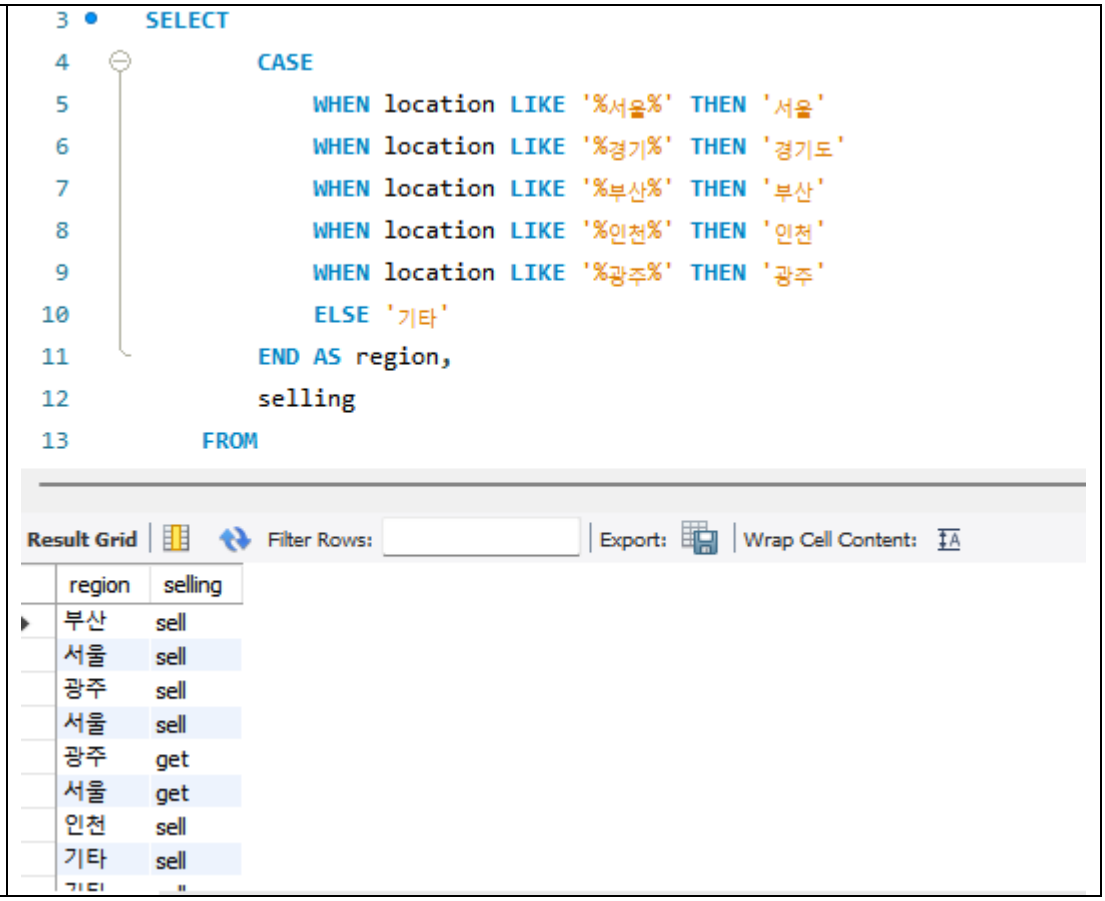
SQL 종류	IN
SQL 사용 위치	상품 개별 조회
SQL 사용 설명	상품 게시글 조회 시, 키워드 기반 추천 상품을 가져오는 SQL문으로 현재 상품의 키워드와 하나라도 겹치면, 즉 현재 상품에 키워드 중에 하나라도 속한다면 그 상품 정보를 추천 상품으로 가져옵니다.
사용한 SQL 문	SELECT DISTINCT sub.*, CASE WHEN ? IS NOT NULL AND EXISTS (SELECT 1 FROM wishlist w WHERE w.product_idx = sub.product_idx AND w.user_idx = ?) THEN TRUE ELSE FALSE

	END AS is_hearted FROM (SELECT p.*, k.keyword FROM product p JOIN keyword k ON p.product_idx = k.product_idx WHERE k.keyword IN (%s) AND p.product_idx != ? AND p.status = 'active') AS sub;
데이터베이스 실행 화면	

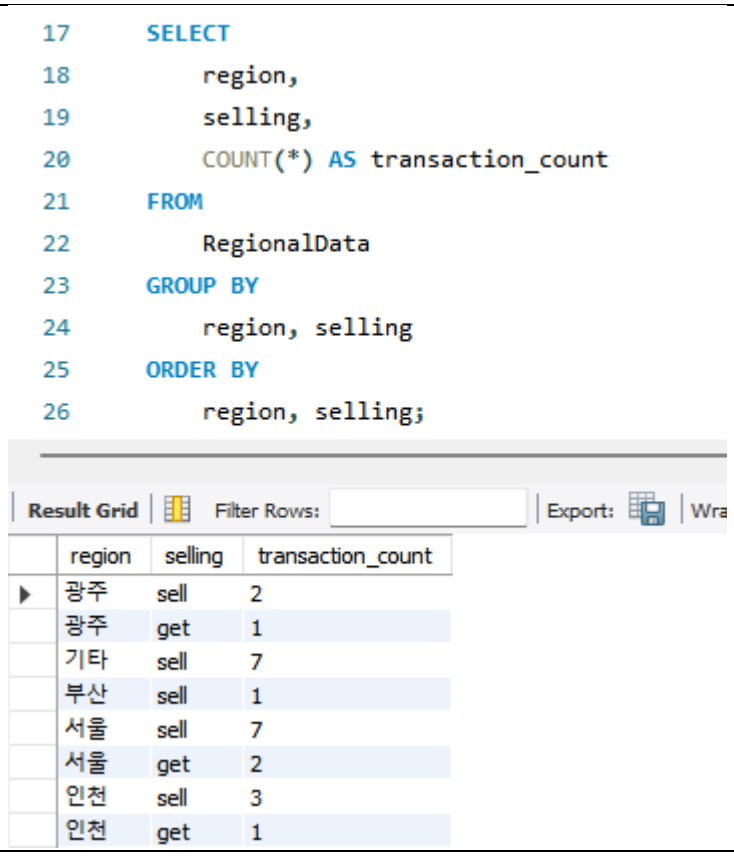
SQL 종류	DISTINCT
SQL 사용 위치	상품 개별 조회
SQL 사용 설명	상품 게시글 조회 시 추천 상품을 가져올 때, 만약 현재 상품과 어느 특정 상품이 키워드가 여러 개 겹친다면, 결과를 중복으로 가져올 수도 있습니다. 그걸 방지하기 위해 DISTINCT를 사용하여 조건을 만족하는 product 튜플을 하나만 가져옵니다.
사용한 SQL 문	SELECT DISTINCT sub.*, CASE WHEN ? IS NOT NULL AND EXISTS (SELECT 1 FROM wishlist w WHERE w.product_idx = sub.product_idx AND w.user_idx = ?) THEN TRUE ELSE FALSE END AS is_hearted FROM (SELECT p.*, k.keyword FROM product p JOIN keyword k ON p.product_idx = k.product_idx WHERE

	k.keyword IN (%s) AND p.product_idx != ? AND p.status = 'active') AS sub;
데이터베이스 실행 화면	

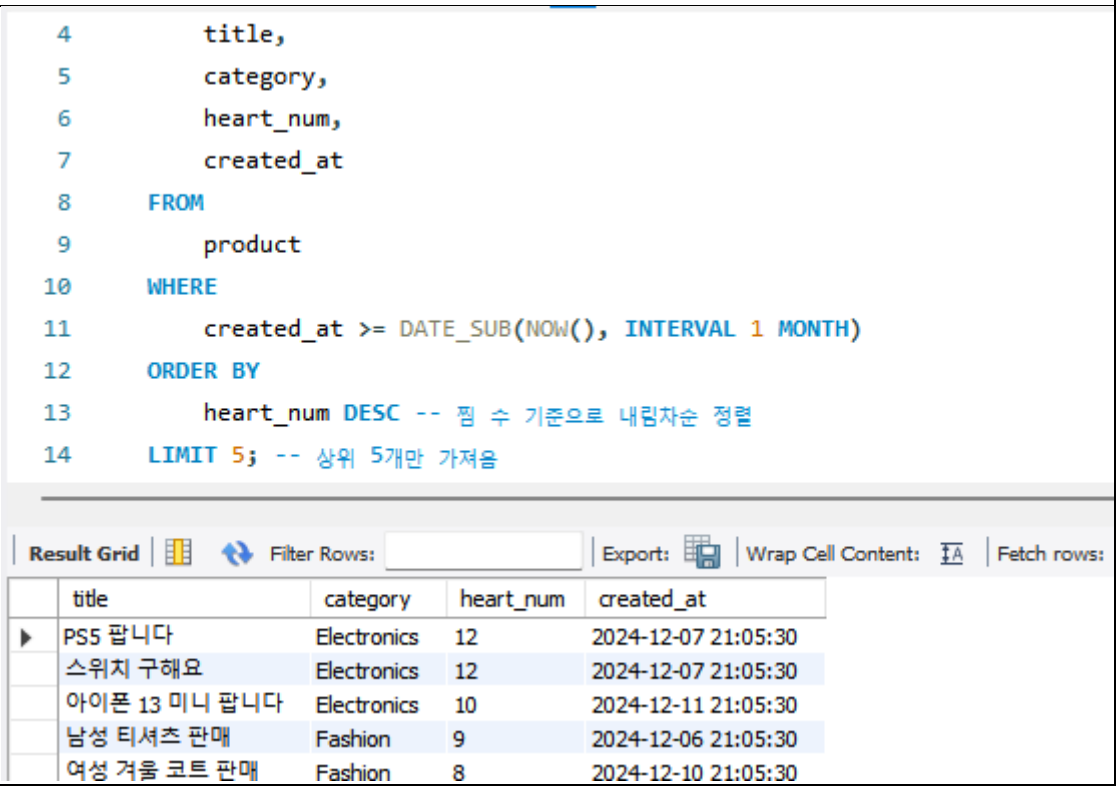
SQL 종류	LIKE
SQL 사용 위치	주요 도시별 거래 비율 데이터 시각화
SQL 사용 설명	데이터 시각화-주요 도시별 거래 비율은 주요 도시별 '팔아요/구해요' 거래 상황을 시각적으로 비교하기 위한 기능입니다. 지역명은 같은 지역에 대해서도 여러 이름으로 저장될 수 있습니다. 예를 들어 "서울", "서울시", "서울특별시"가 모두 가능하므로 핵심 단어인 "서울"을 포함한 모든 데이터를 탐색하기 위해 LIKE로 주요 도시 이름 패턴을 가졌는지 문자열을 비교합니다.
사용한 SQL 문	SELECT CASE WHEN location LIKE '%서울%' THEN '서울' WHEN location LIKE '%경기%' THEN '경기도' WHEN location LIKE '%부산%' THEN '부산' WHEN location LIKE '%인천%' THEN '인천' WHEN location LIKE '%광주%' THEN '광주' ELSE '기타' END AS region, selling FROM product

데이터베이스 실행 화면	

SQL 종류	WITH
SQL 사용 위치	주요 도시별 거래 비율 데이터 시각화
SQL 사용 설명	주요 도시별 거래 비율 데이터를 시각화하는 기능에서 WITH를 사용해 product 테이블에서 지역과 거래 상태를 계산한 결과를 임시 테이블로 저장 후 활용할 수 있도록 합니다.
사용한 SQL 문	WITH RegionalData AS (SELECT CASE WHEN location LIKE '%서울%' THEN '서울' WHEN location LIKE '%경기%' THEN '경기도' WHEN location LIKE '%부산%' THEN '부산' WHEN location LIKE '%인천%' THEN '인천' WHEN location LIKE '%광주%' THEN '광주' ELSE '기타' END AS region, selling FROM product) SELECT region, selling, COUNT(*) AS transaction_count

	FROM RegionalData GROUP BY region, selling ORDER BY region, selling;
데이터베이스 실행 화면	

SQL 종류	LIMIT
SQL 사용 위치	최근 한 달 간 찜을 많이 받은 상품 TOP 5
SQL 사용 설명	최근 한 달 간 가장 많은 찜을 받은 상품 TOP 5를 시각화하는 기능에서 조건에 부합하는 데이터 중 찜 수로 정렬한 후 LIMIT를 이용하여 최상위 결과 5개만 가져옵니다.
사용한	SELECT title, category,

SQL문	heart_num, created_at FROM product WHERE created_at >= DATE_SUB(NOW(), INTERVAL 1 MONTH) ORDER BY heart_num DESC -- 찜 수 기준으로 내림차순 정렬 LIMIT 5; -- 상위 5개만 가져옴
데이터베이스실행화면	

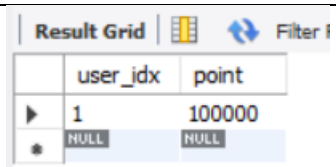
```
4          title,
5          category,
6          heart_num,
7          created_at
8      FROM
9          product
10     WHERE
11         created_at >= DATE_SUB(NOW(), INTERVAL 1 MONTH)
12     ORDER BY
13         heart_num DESC -- 찜 수 기준으로 내림차순 정렬
14     LIMIT 5; -- 상위 5개만 가져옴
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

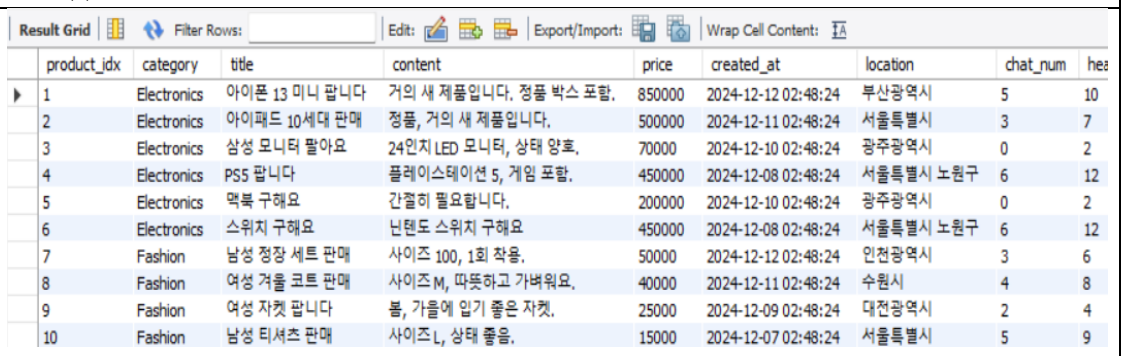
title	category	heart_num	created_at
PS5 팝니다	Electronics	12	2024-12-07 21:05:30
스위치 구해요	Electronics	12	2024-12-07 21:05:30
아이폰 13 미니 팝니다	Electronics	10	2024-12-11 21:05:30
남성 티셔츠 판매	Fashion	9	2024-12-06 21:05:30
여성 겨울 코트 판매	Fashion	8	2024-12-10 21:05:30

SQL종류	SELECT
SQL사용위치	내 정보 가져오기(내 정보 수정에 있어야 하는 정보 가져오기)
SQL사용설명	이 SQL은 user 테이블에서 사용자의 ID를 조건으로 해당 사용자의 모든 정보를 가져옵니다. id는 사용자를 고유하게 식별하는 값으로, WHERE 조건에 사용됩니다. 조회된 데이터를 통해 사용자의 기존 이름, 닉네임, 전화번호, 지역, 프로필 이미지, 소개글 등을 화면에 표시합니다. 사용자에게 필요한 필드를 미리 채워 수정 가능한 상태로 제공합니다.
사용	SELECT * FROM user

한 SQL 문	WHERE user_idx= ?
데이터베이스 실행 화면	

SQL 종류	DUPLICATE
SQL 사용 위치	포인트 충전 및 인출 기능
SQL 사용 설명	INSERT ... ON DUPLICATE KEY UPDATE를 사용하여 포인트 데이터를 효율적으로 관리합니다. point 테이블에 user_idx를 기준으로 데이터를 삽입하거나, 기존 데이터가 있을 경우 해당 사용자의 포인트를 누적(point + ?)합니다. 이 방식으로 새로운 사용자는 자동으로 레코드가 생성되고, 기존 사용자는 포인트가 업데이트됩니다.
사용한 SQL 문	INSERT INTO point (user_idx, point) VALUES ((SELECT user_idx FROM user WHERE id = ?), ?) ON DUPLICATE KEY UPDATE point = point + ?
데이터베이스 실행 화면	

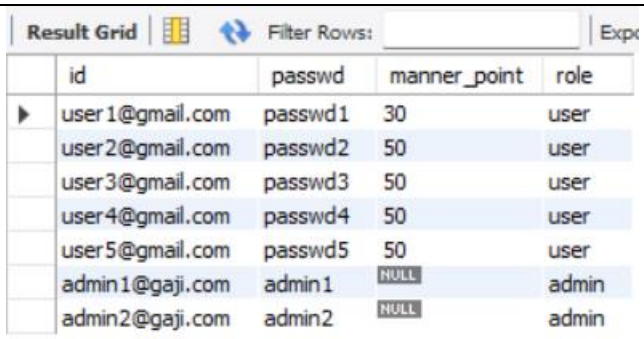
SQL 종류	TRIGGER
SQL 사용 위치	해당 상품에 대해 새로운 채팅이 발생하면, 해당 상품의 채팅 수 증가시키기
SQL 사용 설명	AFTER INSERT 트리거는 Chat 테이블에 데이터가 삽입된 후 자동으로 실행됩니다. 새로운 채팅이 해당 상품에서 첫 번째 대화라면, Product 테이블의 chat_num을 증가시킵니다. 조건문(IF NOT EXISTS)을 사용하여 기존에 동일한 상품과 동일한 사용자 쌍의 대화가 이미 존재하는지 확인합니다. 조건이 만족될 경우에만 chat_num을 증가시킵니다.
사	CREATE TRIGGER update_chat_num

용한 SQL 문	AFTER INSERT ON Chat FOR EACH ROW BEGIN IF NOT EXISTS (SELECT 1 FROM Chat WHERE ((sender_idx = NEW.sender_idx AND receiver_idx = NEW.receiver_idx) OR (sender_idx = NEW.receiver_idx AND receiver_idx = NEW.sender_idx)) AND product_idx = NEW.product_idx AND chat_idx < NEW.chat_idx) THEN -- chat_num 증가 UPDATE Product SET chat_num = chat_num + 1 WHERE product_idx = NEW.product_idx; END IF; END$$
데이터베이스 실행 화면	

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

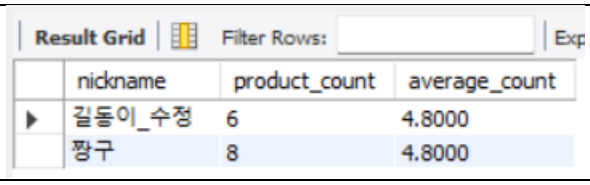
product_idx	category	title	content	price	created_at	location	chat_num	hea
1	Electronics	아이폰 13 미니 팝니다	거의 새 제품입니다. 정품 박스 포함.	850000	2024-12-12 02:48:24	부산광역시	5	10
2	Electronics	아이패드 10세대 판매	정품, 거의 새 제품입니다.	500000	2024-12-11 02:48:24	서울특별시	3	7
3	Electronics	삼성 모니터 팔아요	24인치 LED 모니터, 상태 양호.	70000	2024-12-10 02:48:24	광주광역시	0	2
4	Electronics	PS5 팝니다	플레이스테이션 5, 게임 포함.	450000	2024-12-08 02:48:24	서울특별시 노원구	6	12
5	Electronics	맥북 구해요	간절히 필요합니다.	200000	2024-12-10 02:48:24	광주광역시	0	2
6	Electronics	스위치 구해요	닌텐도 스위치 구해요	450000	2024-12-08 02:48:24	서울특별시 노원구	6	12
7	Fashion	남성 정장 세트 판매	사이즈 100, 1회 착용.	50000	2024-12-12 02:48:24	인천광역시	3	6
8	Fashion	여성 겨울 코트 판매	사이즈 M, 따뜻하고 가벼워요.	40000	2024-12-11 02:48:24	수원시	4	8
9	Fashion	여성 자켓 팝니다	봄, 가을에 입기 좋은 자켓.	25000	2024-12-09 02:48:24	대전광역시	2	4
10	Fashion	남성 티셔츠 판매	사이즈 L, 상태 좋음.	15000	2024-12-07 02:48:24	서울특별시	5	9

SQL 종류	VIEW
SQL 사용 위치	로그인 시, user 테이블과 admin 테이블 한 번에 탐색.
SQL 사용 설명	VIEW는 데이터베이스에서 복잡한 쿼리를 단순화하기 위해 사용됩니다. 뷰는 가상 테이블로, 생성 후 일반 테이블처럼 조회할 수 있지만 실제 데이터를 저장하지 않습니다. 로그인 과정에서 두 테이블(user, admin)을 통합하여 단일 접근 지점 제공. 뷰를 사용함으로써 원본 테이블에 대한 직접 접근을 줄이고 쿼리 재사용성을 높입니다.
사용한 SQL 문	CREATE OR REPLACE VIEW UserAndAdmin AS SELECT id, passwd, manner_point, 'user' AS role FROM user UNION SELECT id, passwd, NULL AS manner_point, 'admin' AS role FROM admin;

데이터베이스 실행 화면	

Result Grid | Filter Rows: | Exp

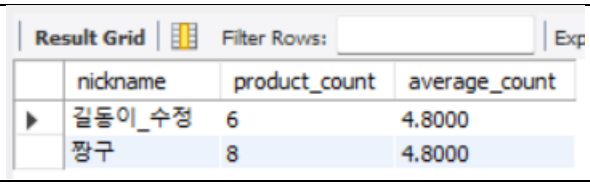
id	passwd	manner_point	role
user1@gmail.com	passwd1	30	user
user2@gmail.com	passwd2	50	user
user3@gmail.com	passwd3	50	user
user4@gmail.com	passwd4	50	user
user5@gmail.com	passwd5	50	user
admin1@gaji.com	admin1	NULL	admin
admin2@gaji.com	admin2	NULL	admin

SQL 종류	CROSS JOIN
SQL 사용 위치	사용자 평균 판매 개수보다 많이 판매하는 사람과 그 개수를 파악하는 데이터 시각화
SQL 사용 설명	CROSS JOIN은 AverageCount(평균 판매 개수)와 ProductCounts(개별 사용자 판매 개수)를 결합하여, 모든 행 조합을 생성하고 조건을 통해 원하는 데이터를 필터링할 때 사용됩니다. CROSS JOIN은 두 테이블의 모든 행의 조합(카티션 곱)을 생성합니다. 이 예제에서는 평균 판매 개수를 담고 있는 AverageCount와 개별 사용자 판매 개수를 담고 있는 ProductCounts를 결합하여 모든 사용자와 평균 판매 개수를 매칭합니다. 평균 판매 개수를 모든 사용자 판매 데이터와 결합하여, 조건(HAVING pc.product_count > ac.average_count)을 통해 평균보다 많이 판매한 사용자만 필터링합니다.
사용한 SQL 문	WITH ProductCounts AS (SELECT writer_idx, COUNT(*) AS product_count FROM Product GROUP BY writer_idx), AverageCount AS (SELECT AVG(product_count) AS average_count FROM ProductCounts) SELECT u.nickname, pc.product_count, ac.average_count FROM User u JOIN ProductCounts pc ON u.user_idx = pc.writer_idx CROSS JOIN AverageCount ac HAVING pc.product_count > ac.average_count;
데이터베이스 실행 화면	

Result Grid | Filter Rows: | Exp

nickname	product_count	average_count
길동이_수정	6	4.8000
짱구	8	4.8000

SQL 종류	HAVING
SQL 사용 위치	사용자 평균 판매 개수보다 많이 판매하는 사람과 그 개수를 파악하는 데이터 시각화
SQL 사용 설명	HAVING 절은 집계 함수(예: COUNT, AVG) 결과를 기반으로 데이터를 필터링하는 데 사용됩니다. 이 예제에서는 평균 판매 개수보다 많은 상품을 판매한 사용자를 필터링하는

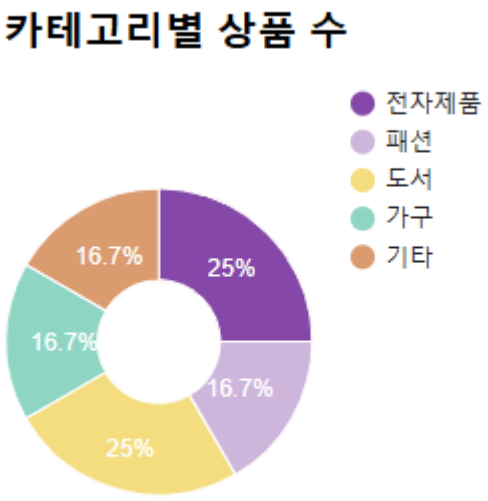
	데 사용됩니다. HAVING 절은 집계 함수 결과를 기준으로 조건을 설정하여 데이터를 필터링합니다. 집계 함수 결과는 WHERE 절에서 사용할 수 없기 때문에, HAVING이 필터링에 사용됩니다. 평균 판매 개수(ac.average_count)와 개별 판매 개수(pc.product_count)를 비교하여, 특정 조건을 만족하는 사용자만 결과로 출력합니다. 이 과정에서, 필터링 조건은 HAVING 절에 정의됩니다.
사용한 SQL 문	WITH ProductCounts AS (SELECT writer_idx, COUNT(*) AS product_count FROM Product GROUP BY writer_idx), AverageCount AS (SELECT AVG(product_count) AS average_count FROM ProductCounts) SELECT u.nickname, pc.product_count, ac.average_count FROM User u JOIN ProductCounts pc ON u.user_idx = pc.writer_idx CROSS JOIN AverageCount ac HAVING pc.product_count > ac.average_count;
데이터베이스 실행 화면	

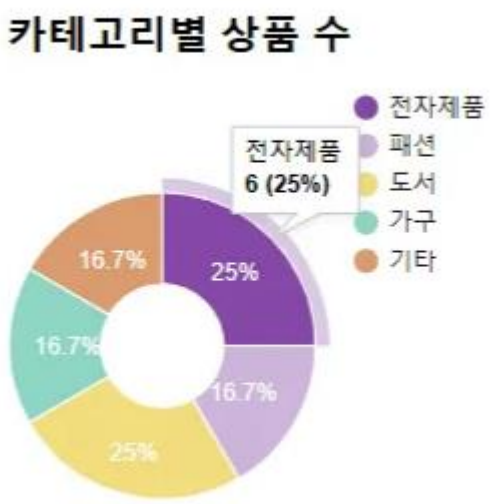
SQL 종류	UNION
SQL 사용 위치	로그인 시, user 테이블과 admin 테이블 한 번에 탐색.
SQL 사용 설명	UNION은 서로 다른 두 개 이상의 SELECT 결과를 하나의 결과로 결합할 때 사용됩니다. 이 예제에서는 user 테이블과 admin 테이블의 데이터를 결합하여 사용자와 관리자를 통합적으로 조회할 수 있는 뷰를 생성합니다. UNION은 두 SELECT 문의 결과를 병합하여 하나의 결과 집합으로 반환합니다. 중복된 데이터를 자동으로 제거합니다. 이 예제에서는 user 테이블과 admin 테이블의 공통 필드(id, passwd)를 결합하고, 각 데이터의 출처를 구분하기 위해 role 필드를 추가합니다. UNION은 데이터 구조가 비슷하지만 다른 테이블의 데이터를 하나의 결과로 결합해야 할 때 유용합니다.
사용한 SQL 문	CREATE OR REPLACE VIEW UserAndAdmin AS SELECT id, passwd, manner_point, 'user' AS role FROM user UNION SELECT id, passwd, NULL AS manner_point, 'admin' AS role FROM admin;

데이터베이스 실행 화면	Result Grid | Filter Rows: | Exp id | passwd | manner_point | role user1@gmail.com | passwd1 | 30 | user user2@gmail.com | passwd2 | 50 | user user3@gmail.com | passwd3 | 50 | user user4@gmail.com | passwd4 | 50 | user user5@gmail.com | passwd5 | 50 | user admin1@gaji.com | admin1 | NULL | admin admin2@gaji.com | admin2 | NULL | admin

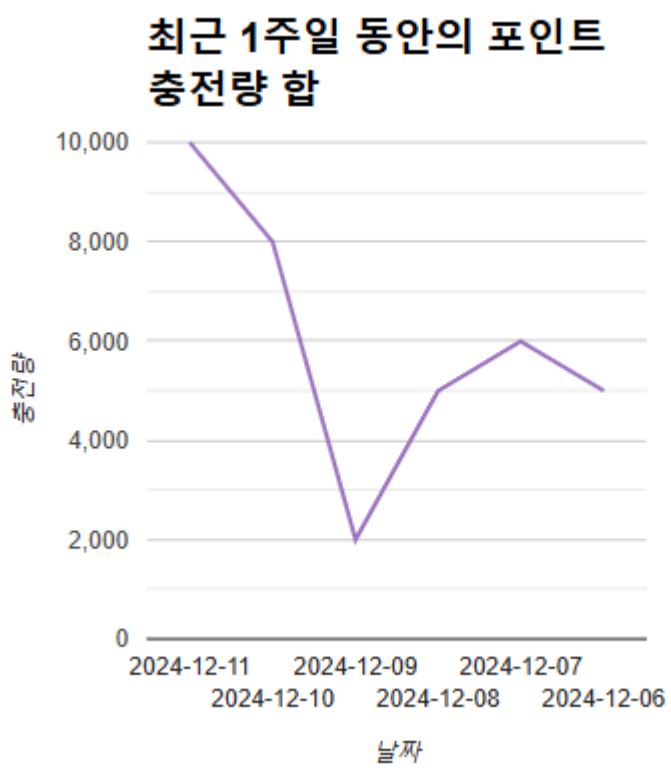
1. 데이터 시각화

모든 데이터 시각화는 Google Chart를 이용하여 구현하며 관리자로 로그인 후 관리자용 메인 페이지에서 확인 가능합니다.

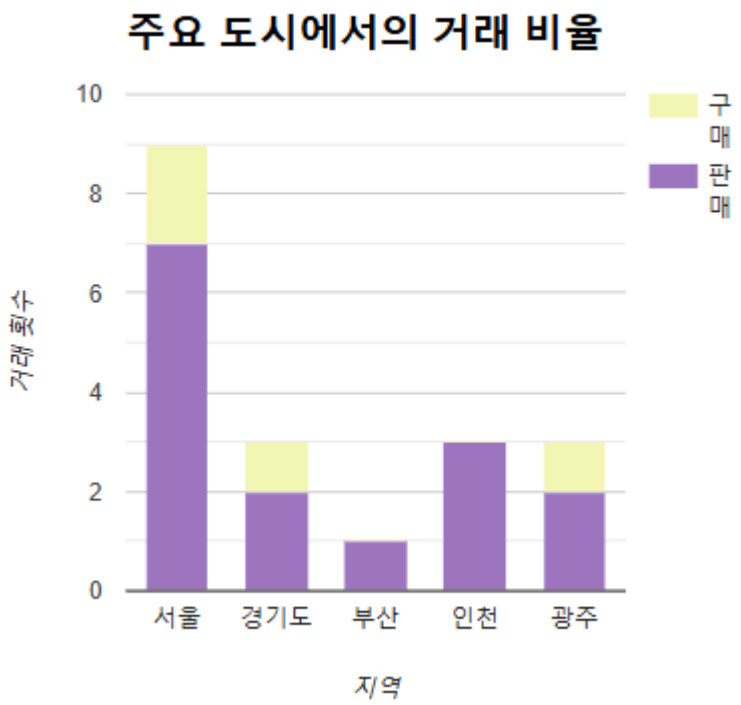
데이터 시각화 주제	**카테고리별 상품 수**
데이터 시각화 설명	카테고리별로 등록된 상품 수를 기반으로 차트를 생성합니다. 관리자는 각 카테고리에서 얼마나 많은 상품이 등록되었는지를 시각적으로 확인할 수 있습니다. 각 카테고리 별 상품 수 비율을 쉽게 비교할 수 있어 비즈니스 인사이트를 제공하며, 관리자가 특정 카테고리에 대한 데이터 분석이 가능하게 합니다.
사용한 SQL 코드	SELECT category, COUNT(*) AS product_count FROM product GROUP BY category
데이터 시각화 차트 설명	product 테이블에서 카테고리를 기준으로 그룹화하여 각 그룹별 속한 product의 수를 셉니다. 선택된 데이터를 원형 차트로 만들어 한 눈에 각 비율을 확인할 수 있도록 합니다.
데이터 시각화 결과 화면	

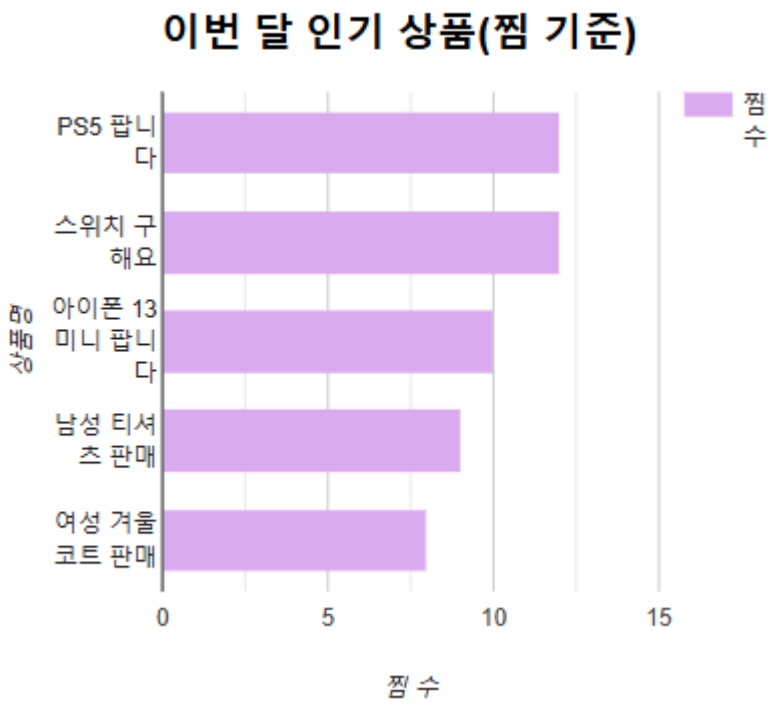
데이터 시각화 주제	**최신 일주일 간 포인트 충전량 합**
데이터 시각화 설명	최근 일주일 동안의 포인트 충전량을 시각적으로 확인할 수 있도록 돕는 기능입니다. 일별, 일주일 간의 포인트 충전 데이터 변화를 직관적으로 제공합니다. 시간 흐름에 따라 포인트 충전량이 어떻게 변화하는지를 시각화할 수 있도록 지원합니다. 시간 경과에 따른 사용자의 포인트 충전 패턴 분석에 유용합니다. 데이터 기반으로 유효한 인사이트 제공이 가능하며, 플랫폼의 성장 추이를 확인할 수 있습니다.
사용한 SQL 코드	SELECT DATE(created_at) AS date, SUM(change_amount) AS total_charge FROM Point_history WHERE transaction_type = 'charge' AND created_at >= CURDATE() - INTERVAL 7 DAY GROUP BY DATE(created_at) ORDER BY DATE(created_at) DESC;
데이터 시각화 차트 설명	point_history 테이블에서 작업 유형이 '충전'이고, 기록 생성 일자가 7일 내인 데이터를 작성일자 별로 그룹화하고, 최신순으로 정렬하여 날짜와 날짜별 포인트 충전량 합을 구합니다.

데이터 시각화 결과 화면	

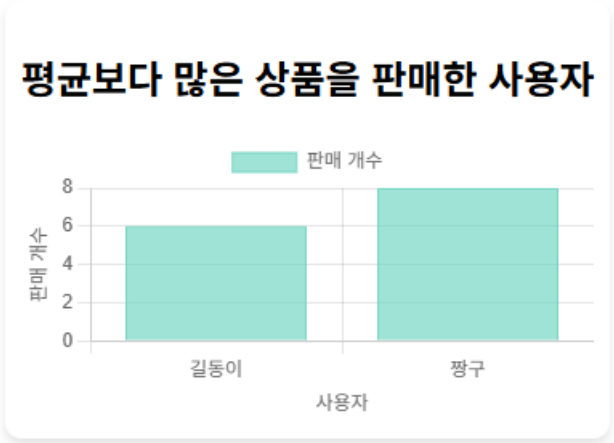
데이터 시각화 주제	주요 도시별 거래 타입 비율
데이터 시각화 설명	특정 도시에서의 거래 유형을 시각적으로 표현하여 관리자가 각 도시에서 발생하는 거래의 주요 경향을 쉽게 파악할 수 있도록 합니다. 주요 도시는 서울특별시, 경기도 지역, 부산광역시, 인천광역시, 광주광역시로 설정했습니다. 도시별 거래 유형 비율을 막대 그래프로 시각화하여 사용자가 직관적으로 이해할 수 있도록 제공합니다. 도시별로 발생하는 거래 유형의 상대적 비율을 시각적으로 표현하여 사용자 이해를 돕습니다. 특정 도시에서의 특정 거래 유형의 빈도수나 비율 변화를 한눈에 볼 수 있어 분석이 용이하고, 도시 간 비교나 특정 지역의 거래 패턴을 이해하는 데 유용합니다.
사용한 SQL 코드	WITH RegionalData AS (SELECT CASE WHEN location LIKE '%서울%' THEN '서울' WHEN location LIKE '%경기%' THEN '경기도' WHEN location LIKE '%부산%' THEN '부산' WHEN location LIKE '%인천%' THEN '인천' WHEN location LIKE '%광주%' THEN '광주' ELSE '기타' END AS region, selling FROM product) SELECT region, selling,

	COUNT(*) AS transaction_count FROM RegionalData GROUP BY region, selling ORDER BY region, selling;
데이터 시각화 차트 설명	product에서 '팔아요/구해요'를 구분하는 속성 selling 값을 기준으로 판매(sell)과 구매(get) 수를 COUNT를 이용해 계산합니다. 지역명이 다양한 경우를 처리하기 위해 LIKE와 CASE를 이용하여 주요 도시를 통합합니다.
데이터 시각화 결과 화면	

데이터 시각화 주제	**최근 한 달 간 찜을 많이 받은 상품 TOP 5**
데이터 시각화 설명	최근 한 달 동안 업로드 된 상품 중에서 가장 많이 찜(좋아요 또는 관심 표시)을 받은 상품 TOP 5를 시각화하여 관리자가 인기 있는 상품을 한눈에 확인할 수 있도록 합니다. 사용자에게 가장 인기 있는 상품을 직관적으로 보여주어 구매 및 관심 표시 패턴을 파악할 수 있게 하고, 트렌드를 파악하고 상품 기획/마케팅에 활용 가능합니다.
사용한 SQL 코드	SELECT title, category, heart_num, created_at FROM product WHERE created_at >= DATE_SUB(NOW(), INTERVAL 1 MONTH) ORDER BY

	heart_num DESC -- 찜 수 기준으로 내림차순 정렬 LIMIT 5; -- 상위 5개만 가져옴
데이터 시각화 차트 설명	product 테이블에서 생성 일자가 현재 날짜로부터 30일 이내인 상품들에서 찜 수를 기준으로 정렬한 결과에서 상위 5개만 선택합니다.
데이터 시각화 결과 화면	

데이터 시각화 주제	**사용자 평균 판매 개수보다 많이 판매하는 사람과 그 개수**
데이터 시각화 설명	이 데이터 시각화는 플랫폼 내 사용자가 평균적으로 판매하는 상품 개수를 기준으로, 그보다 많이 판매한 사용자와 해당 판매 개수를 나타냅니다. 우수 판매자를 파악하고, 플랫폼의 사용자 활동 패턴을 분석하는데 목적이 있습니다. 핵심 데이터로는 사용자 닉네임 (nickname), 판매 개수 (product_count), 전체 평균 판매 개수 (average_count)가 있습니다.
사용한 SQL 코드	WITH ProductCounts AS (SELECT writer_idx, COUNT(*) AS product_count FROM Product GROUP BY writer_idx), AverageCount AS (SELECT AVG(product_count) AS average_count FROM ProductCounts) SELECT u.nickname, pc.product_count, ac.average_count FROM User u JOIN ProductCounts pc ON u.user_idx = pc.writer_idx CROSS JOIN AverageCount ac HAVING pc.product_count > ac.average_count;
데이터 시각화	차트 유형은 막대 차트입니다.

차트 설명	• x축: 사용자 닉네임 (nickname) • y축: 판매 개수 (product_count) 각 막대는 평균보다 많은 상품을 판매한 사용자의 판매 개수를 나타냅니다.
데이터 시각화 결과 화면	

2. YouTube 및 GitHub 링크

※ 성적 마감시까지 공개로 설정

YouTube 링크	
Github 링크	https://github.com/gitSoyoungLee/Gaji-Market.git

3. 고찰

이름	**이소영**
기여 사항	- 프로젝트 초기 기획 및 요구사항 분석과 정의 - 웹 애플리케이션 디자인 설계 - 데이터베이스 설계 - API 설계 및 구현 - 팀 협업 및 코드 리뷰 적극 참여
담당 구현한 주요 기능	담당한 기능에 대해서 백엔드와 프론트엔드 사이드를 구현했습니다. - 상품 목록 조회 기능 - 상품 개별 조회 기능 - 상품 후기 작성 및 조회 기능 - 상품 게시글 등록, 수정, 삭제 기능 - 상품 찜 등록, 해제 기능 - 문의 게시판 목록 조회, 문의 게시글 등록, 답글 등록 기능 - 신고, 신고 조회, 신고 처리 기능 - 관리자 전용 기능: 회원 관리, 관리자 관리 기능 - 데이터 시각화: 카테고리별 상품 수, 최신 일주일 포인트 충전량 합, 주요 도시별 거래 타입 비율, 최근 한 달 간 찜을 많이 받은 상품 TOP 5
자기 평가	프로젝트 초기 기획 단계부터 개발 및 테스트 단계에 모두 적극적으로 참여하며 프로젝트 목표를 달성하고자 노력했습니다. 특히 실제 사용자의 입장을 고려하며 백엔드, 프론트엔드 개발 시 사용자 경험을 개선하는 기능을 구현하는 데 중점을 두었습니다. 다른 팀원과 정기적으로 회의하며 협동 프로젝트에 적극적으로 참여했고, 데이터베이스 설계와 구조 강의에서 배운 지식을 바탕으로 설계와 구현하고자 노력하며 여러 의견을 주도적으로 건의했습니다.

이름	**한은진**
기여 사항	- 프로젝트 초기 기획 및 요구사항 분석과 정의

	- 프로젝트 전체 일정 관리 - 데이터베이스 설계 - API 설계 및 구현 - 팀 협업 및 보고서 작성 적극 참여
담당 구현한 주요 기능	담당한 기능에 대해서 백엔드와 프론트엔드 사이드를 구현했습니다. - 로그인, 로그아웃 및 회원가입 기능 - 마이페이지(판매 내역, 구매 내역, 찜 내역, 문의 내역) 기능 - 관리자 매너지수 설정 기능 - 내 정보 수정 기능 - 포인트 충전 및 인출 기능 - 본인 인증 기능 - 타 판매자 프로필 가져오기 기능 - 상품 검색 기능 - 채팅 기능 - 채팅 중 송금 기능 - 공지 기능 - 관리자가 공지 작성, 수정 및 삭제 기능 - 중요 공지 설정 기능 - 데이터 시각화: 사용자 평균 판매 개수보다 많이 판매하는 사람과 그 개수
자기 평가	프로젝트의 초기 기획 단계부터 요구사항 분석 및 정의를 주도적으로 수행하며 프로젝트의 방향성을 올바르게 설정하도록 노력했습니다. 프로젝트 일정 관리와 데이터베이스 설계, API 설계 및 구현을 팀원과 함께 진행하여 전체적인 구조와 로직의 안정성을 확보하는 데 중점을 두었습니다. 또한, 프로젝트 기간 동안 정기적인 회의를 진행하여 팀원 간의 의사소통을 원활하게 할 수 있었습니다. 이번 프로젝트를 통해 기술적인 역량뿐만 아니라 프로젝트 관리 능력, 협업 능력, 문제 해결 능력을 키울 수 있었습니다. 특히, 데이터베이스 설계와 API 구현에서의 꼼꼼한 접근과 채팅 중 송금과 같은 복합 기능 구현 경험은 향후 프로젝트에서도 큰 자산이 될 것으로 생각합니다.